TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



CÔNG NGHỆ PHẦN MỀM

Bài tập lớn – Assignment 2

BrewLite

Hệ thống đặt cà phê mobile-first/PWA
và thanh toán mô phỏng

REQUIREMENTS TO DEPLOYMENT

GVHD: Từ Lãng Phiêu
SV: Phạm Duy Đăng - 3124410068
Lớp: DCT1245 - K24
Ngành: Công nghệ Thông tin

TP. HỒ CHÍ MINH, THÁNG 11/2026

Thông tin sinh viên và phân công

Đề tài được thực hiện bởi một sinh viên, đảm nhận toàn bộ các vai trò của nhóm phát triển.

STT	Họ và tên	MSSV	Lớp	Vai trò / nhiệm vụ	Hoàn thành
1	Phạm Duy Đăng	3124410068	DCT1245	Product Owner, phân tích yêu cầu, thiết kế, Backend NestJS, Frontend Next.js/PWA, kiểm thử, DevOps (CI/CD, triển khai)	100%

Khóa – Ngành: K24 – Công nghệ Thông tin

Email liên hệ: duydang.contact@gmail.com

Repository: https://github.com/dangpham-work/CupPulse

Lịch sử phiên bản tài liệu

Phiên bản	Ngày	Nội dung thay đổi	Người thực hiện
1.0	01/11/2026	Task 1 – Khai thác và đặc tả yêu cầu (bối cảnh, stakeholder, phạm vi, SRS, use-case)	Phạm Duy Đăng
1.1	02/11/2026	Task 2 – Mô hình hóa hệ thống (activity, sequence, class, state diagram); ma trận truy vết v0.2	Phạm Duy Đăng
1.2	02/11/2026	Task 3 – Thiết kế kiến trúc (C4, 9 ADR, deployment diagram); ma trận truy vết v0.3	Phạm Duy Đăng
1.3	02/11/2026	Task 4 – Sprint 1: repository, quy trình, MVP xem menu, CI; ma trận truy vết v0.4	Phạm Duy Đăng

Mục lục

Thông tin sinh viên và phân công **1**

Lịch sử phiên bản tài liệu **1**

1 Tổng quan đề tài **6**
1.1 Giới thiệu . . . 6
1.2 Quy trình thực hiện . . . 6
1.3 Kho mã nguồn và tài liệu . . . 6

2 Task 1 – Khai thác và đặc tả yêu cầu **8**
2.1 Task 1.1 – Bối cảnh, stakeholder và phạm vi . . . 8
2.1.1 Bối cảnh và vấn đề nghiệp vụ . . . 8
2.1.2 Stakeholder . . . 8
2.1.3 Phạm vi . . . 10
2.1.4 Giả định và ràng buộc . . . 11
2.1.5 Rủi ro ban đầu . . . 11
2.2 Task 1.2 – Yêu cầu chức năng và phi chức năng (SRS) . . . 13
2.2.1 Quy ước . . . 13
2.2.2 Tác nhân và vai trò . . . 13
2.2.3 Quy tắc nghiệp vụ . . . 13
2.2.4 Yêu cầu chức năng . . . 14
2.2.5 Yêu cầu phi chức năng . . . 18
2.2.6 Sơ đồ use-case tổng quát . . . 21
2.2.7 User story . . . 22
2.2.8 Giao diện ngoài . . . 23
2.3 Task 1.3 – Đặc tả use-case trọng tâm: UC-01 Đặt và thanh toán đơn hàng . . . 24
2.3.1 Luồng chính . . . 24
2.3.2 Luồng thay thế . . . 25
2.3.3 Luồng ngoại lệ . . . 26
2.3.4 Quy tắc đặc biệt . . . 26
2.3.5 Liên kết use-case – yêu cầu – tiêu chí chấp nhận . . . 27
2.3.6 Kịch bản kiểm thử nghiệm thu . . . 27

3 Task 2 – Mô hình hóa hệ thống **28**
3.1 Task 2.1 – Activity diagram . . . 28
3.2 Task 2.2 – Sequence diagram . . . 31
3.3 Task 2.3 – Class diagram . . . 34
3.4 Sơ đồ trạng thái bổ trợ . . . 35
3.5 Kiểm tra nhất quán với SRS và use-case . . . 36

4 Task 3 – Thiết kế kiến trúc **37**
4.1 Task 3.1 – Phương pháp và quyết định kiến trúc . . . 37
4.1.1 Phương pháp . . . 37
4.1.2 Động lực kiến trúc . . . 39
4.1.3 Các container và thành phần . . . 39
4.1.4 Quyết định kiến trúc (ADR) . . . 40
4.1.5 Lý do chọn, ưu điểm và hạn chế . . . 41
4.2 Task 3.2 – Deployment diagram . . . 42

5 Task 4 – Triển khai Sprint 1 **45**
5.1 Task 4.1 – Thiết lập repository . . . 45
5.2 Task 4.2 – Quản lý tài liệu . . . 45
5.3 Task 4.3 – MVP Sprint 1 . . . 46
5.4 Task 4.4 – Continuous Integration . . . 47

6 Task 5 – Triển khai Sprint 2, CI/CD và Delivery **49**

7 Ma trận truy vết **49**

Tài liệu tham khảo **50**

Danh sách hình vẽ

1 Sơ đồ use-case tổng quát của hệ thống BrewLite . . . 21
2 Use-case diagram cho phạm vi UC-01 . . . 24
3 Activity-01 – Luồng đặt món end-to-end (swimlane) . . . 29
4 Sequence-01 (phần 1/2) – Giai đoạn 1: tạo đơn, kiểm tra JWT, validate, transaction trừ kho 31
5 Sequence-01 (phần 2/2) – Giai đoạn 2–3: thanh toán, idempotency, `alt` thành công/thất bại/timeout, thông báo kết quả . . . 32
6 Class-01 – Mô hình domain BrewLite . . . 34
7 State-01 – Vòng đời Order và Payment . . . 36
8 C4-01 – System Context của BrewLite . . . 37
9 C4-02 – Container của BrewLite . . . 37
10 C4-03 – Component của NestJS API (modular monolith) . . . 38
11 Deployment-01 – Local, CI và Production (Vercel + Render) . . . 43
12 Màn menu ở chiều rộng mobile: dữ liệu từ API (trái) và trạng thái lỗi khi API không phản hồi (phải) . . . 46
13 CI của PR #30 thành công: ba job xanh . . . 48
14 CI của PR #31 phát hiện lỗi lint: job chất lượng và job Docker đỏ . . . 48

Danh sách bảng

1 Các task, tạo phẩm và trạng thái . . . 6
2 Ánh xạ báo cáo – repository . . . 7
3 Vấn đề nghiệp vụ hiện tại . . . 8
4 Mục tiêu đo được . . . 8
5 Bảng stakeholder . . . 8
6 Ma trận quyền lực – quan tâm . . . 9
7 Phạm vi MVP (in-scope, Must) . . . 10
8 Phạm vi mở rộng (Should/Could) . . . 10
9 Ngoài phạm vi (out-of-scope, Won't) . . . 10
10 Giả định . . . 11
11 Ràng buộc . . . 11
12 Rủi ro ban đầu . . . 12
13 Quy ước MoSCoW . . . 13
14 Tác nhân . . . 13
15 Quy tắc nghiệp vụ . . . 13
16 Dữ liệu test của Payment Service mô phỏng . . . 14
17 Tổng hợp yêu cầu chức năng . . . 14
18 Yêu cầu phi chức năng . . . 18
19 Tổng hợp mức ưu tiên . . . 20
20 Danh sách use-case . . . 21
21 User story . . . 22
22 Danh sách API dự kiến . . . 23
23 Thông tin chung UC-01 . . . 24
24 Luồng chính UC-01 . . . 25
25 Luồng thay thế UC-01 . . . 25
26 Luồng ngoại lệ UC-01 . . . 26
27 Liên kết UC-01 với yêu cầu . . . 27
28 Kịch bản nghiệm thu rút từ UC-01 . . . 27
29 Danh sách sơ đồ Task 2 . . . 28
30 Activity-01 đối chiếu yêu cầu đề bài . . . 30
31 Sequence-01 đối chiếu yêu cầu đề bài . . . 33
32 Các lớp domain . . . 34
33 Enum trạng thái . . . 35
34 Đối chiếu nhất quán giữa các sơ đồ và SRS . . . 36
35 Động lực kiến trúc . . . 39

36 Container của BrewLite 39
37 Tóm tắt ADR: lựa chọn và trade-off 40
38 Modular monolith so với microservices 41
39 Truy vết node triển khai → kiến trúc → mã nguồn 42
40 Giao thức, cổng và kết nối 42
41 Secret và biến môi trường 44
42 Yêu cầu Task 4.1 và hiện thực 45
43 Lịch sử thay đổi tài liệu qua pull request 45
44 Phạm vi MVP Sprint 1 46
45 Test tự động Sprint 1 47
46 Các lần chạy CI của Sprint 1 47
47 Ma trận truy vết yêu cầu chức năng 49
48 Ma trận truy vết yêu cầu phi chức năng then chốt 50

1 Tổng quan đề tài

1.1 Giới thiệu

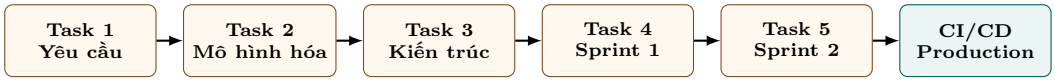
Assignment 2 tiếp tục đề tài **BrewLite** của Assignment 1. Nhóm phát triển dự án theo một chuỗi công việc hoàn chỉnh: khai thác yêu cầu, mô hình hóa, thiết kế kiến trúc, hiện thực qua hai Sprint, thiết lập CI/CD và triển khai hệ thống lên môi trường có thể truy cập được.

Luồng nghiệp vụ tối thiểu của BrewLite:

$$\text{Xem menu} \rightarrow \text{Chọn tùy chọn} \rightarrow \text{Giỏ hàng} \rightarrow \text{Đặt đơn} \rightarrow \text{Thanh toán} \rightarrow \text{Theo dõi đơn}$$

Công nghệ bắt buộc: **Next.js** (mobile-first/PWA) cho frontend, **NestJS** cho backend, **PostgreSQL** để lưu dữ liệu và **Docker** để đóng gói.

1.2 Quy trình thực hiện



Bảng 1: Các task, tạo phẩm và trạng thái

Task	Giai đoạn	Tạo phẩm chính	Điểm	Trạng thái
1	Khai thác và đặc tả yêu cầu	Stakeholder, phạm vi, SRS, use-case tổng quát, đặc tả use-case	2	Hoàn thành
2	Mô hình hóa hệ thống	Activity, sequence, class diagram	2	Hoàn thành
3	Thiết kế kiến trúc	Kiến trúc, ADR, deployment diagram	2	Hoàn thành
4	Triển khai Sprint 1	Repository, tài liệu, CI, MVP xem menu	2	Hoàn thành
5	Sprint 2, CI/CD, Delivery	MVP hoàn chỉnh, CD, production, kiểm thử, bàn giao	2	Chưa thực hiện

1.3 Kho mã nguồn và tài liệu

Toàn bộ tạo phẩm được quản lý phiên bản trong repository https://github.com/dangpham-work/CupPulse. Bảng dưới đây ánh xạ các phần của báo cáo tới tệp tương ứng trong repository.

Bảng 2: Ánh xạ báo cáo – repository

Phần báo cáo	**Tệp trong repository**
Mục 2.1	`docs/requirements/context-and-scope.md`
Mục 2.2	`docs/requirements/srs.md`
Mục 2.3	`docs/requirements/use-case-checkout.md`
Hình 1	`docs/diagrams/usecase-overview.puml`, `.svg`
Hình 2	`docs/diagrams/usecase-checkout.puml`, `.svg`
Hình 3	`docs/diagrams/activity-order-e2e.puml`, `.svg`
Hình 4	`docs/diagrams/sequence-checkout.puml`, `.svg`
Hình 6	`docs/diagrams/class-domain.puml`, `.svg`
Hình 7	`docs/diagrams/state-order-payment.puml`, `.svg`
Mục 3	`docs/diagrams/README.md`
Mục 4	`docs/architecture/architecture.md`, `docs/architecture/adr/00x-*.md`
Hình 8 – 10	`docs/architecture/diagrams/c4-*.puml`, `.svg`
Hình 11	`docs/architecture/diagrams/deployment.puml`, `.svg`; `docs/architecture/deployment-view.md`
Mục 5	`README.md`, `CONTRIBUTING.md`, `docs/process/`, `apps/web`, `apps/api`, `.github/workflows/ci.yml`, `docker-compose.yml`
Mục 7	`docs/traceability-matrix.md`

2 Task 1 – Khai thác và đặc tả yêu cầu

2.1 Task 1.1 – Bối cảnh, stakeholder và phạm vi

2.1.1 Bối cảnh và vấn đề nghiệp vụ

BrewLite là một quán cà phê mang đi quy mô nhỏ (một cửa hàng, 2–4 nhân viên quầy). Hiện quán bán theo cách truyền thống: khách xếp hàng, gọi món bằng lời, nhân viên ghi tay và thu tiền tại quầy. Vào giờ cao điểm (7:00–9:00 và 11:30–13:00) phát sinh các vấn đề trong Bảng 3.

Bảng 3: Vấn đề nghiệp vụ hiện tại

#	Vấn đề hiện tại	Hậu quả
P1	Khách phải xếp hàng để gọi món và trả tiền	Thời gian chờ dài, khách bỏ về, mất doanh thu
P2	Gọi món bằng lời, ghi tay	Sai món, sai size/topping, sai tiền
P3	Không biết món nào sắp hết	Nhận đơn rồi mới báo hết, khách phàn nàn
P4	Khách không biết đơn đã xong chưa	Khách đứng chờ quanh quầy, nhân viên bị hỏi liên tục
P5	Không lưu lịch sử mua hàng	Không chạy được khuyến mãi hay tích điểm để giữ chân khách
P6	Doanh thu và đơn hàng ghi tay	Quản lý khó đối soát và thống kê

Mục tiêu của BrewLite: cho khách tự xem menu, chọn tùy chọn, đặt và thanh toán trên điện thoại (PWA, không cần cài ứng dụng từ store), rồi theo dõi trạng thái đơn. Nhân viên nhận đơn đã thanh toán và cập nhật trạng thái. Quản lý kiểm soát sản phẩm, tồn kho và khuyến mãi.

Bảng 4: Mục tiêu đo được

Mã	Mục tiêu	Chỉ số	Ngưỡng đạt
G1	Giảm thời gian gọi món	Thời gian từ mở app đến thanh toán xong (người dùng mới)	$\leq$ 3 phút
G2	Đơn chính xác	Tỷ lệ đơn sai size/topping so với lựa chọn của khách	0%
G3	Không bán hàng đã hết	Số đơn được tạo khi món đã hết tồn kho	0
G4	Khách biết trạng thái đơn	Thời gian trạng thái mới hiển thị cho khách	$\leq$ 5 giây
G5	Truy cập được để nghiệm thu	URL production HTTPS hoạt động lúc chấm	100%

2.1.2 Stakeholder

Bảng 5: Bảng stakeholder

Mã	Stakeholder	Vai trò	Mục tiêu	Kỳ vọng với hệ thống	Ảnh hưởng
SH-01	**Khách hàng**	Người dùng cuối, đặt và trả tiền	Mua đồ uống nhanh, đúng món, không xếp hàng	Menu rõ giá; chọn size/topping dễ; thanh toán Ví/Thẻ nhanh; biết lúc nào đơn xong; xem lại đơn cũ; được khuyến mãi, tích điểm	Trung bình

Mã	Stakeholder	Vai trò	Mục tiêu	Kỳ vọng với hệ thống	Ảnh hưởng
SH-02	**Nhân viên quầy**	Nhận, chế biến đơn, cập nhật trạng thái	Làm đúng món, ít bị hỏi, thao tác nhanh giờ cao điểm	Chỉ thấy đơn đã thanh toán; chi tiết đơn rõ (size, topping, ghi chú); đổi trạng thái một chạm	Trung bình
SH-03	**Quản lý cửa hàng**	Chủ nghiệp vụ; quản lý menu, kho, khuyến mãi	Tăng doanh thu, kiểm soát tồn kho, giữ chân khách	Sửa sản phẩm, giá, tồn kho; tạo mã khuyến mãi; xem doanh thu; không bán món đã hết	Cao
SH-04	**Giảng viên / khách hàng nghiệm thu**	Đánh giá và nghiệm thu sản phẩm	Kiểm chứng nhóm đi đúng quy trình từ yêu cầu đến triển khai	Tạo phẩm đủ, đúng đường dẫn, truy vết được; CI/CD chạy; URL production hoạt động; clone về chạy được theo README	Rất cao
SH-05	**Nhóm phát triển** (1 SV: Phạm Duy Đăng)	Kiêm mọi vai trò: PO, phân tích, thiết kế, lập trình, kiểm thử, triển khai	Hoàn thành đúng hạn, học quy trình chuẩn	Yêu cầu rõ ràng, ổn định; stack bắt buộc; hạ tầng miễn phí	Cao
SH-06	**Đơn vị thanh toán** (mô phỏng)	Hệ thống ngoài xử lý giao dịch Ví/Thẻ	Giao dịch hợp lệ, không trừ tiền trùng	API rõ ràng; mỗi giao dịch có idempotency key; trả kết quả kèm mã lỗi; không nhận dữ liệu thẻ thừa	Trung bình

Bảng 6: Ma trận quyền lực – quan tâm

	Quan tâm thấp	**Quan tâm cao**
Ảnh hưởng cao	–	SH-03, SH-04, SH-05: quản lý chặt, duyệt mọi thay đổi phạm vi
Ảnh hưởng thấp/TB	SH-06: theo dõi, tuân thủ hợp đồng API	SH-01, SH-02: cập nhật thường xuyên, thử nghiệm khả dụng

2.1.3 Phạm vi

Bảng 7: Phạm vi MVP (in-scope, Must)

#	Hạng mục	Yêu cầu
S1	Xem menu theo danh mục, có trạng thái loading/empty/error	FR-01
S2	Chi tiết sản phẩm: chọn size và topping, giá cập nhật theo lựa chọn	FR-02
S3	Giỏ hàng và tính tổng tiền	FR-03
S4	Đặt đơn: kiểm tra và trừ tồn kho trong một transaction	FR-04
S5	Thanh toán Ví/Thẻ qua dịch vụ thanh toán mô phỏng	FR-05
S6	Idempotency khi client gửi lại yêu cầu thanh toán	FR-06
S7	Xác nhận và theo dõi trạng thái đơn	FR-07
S8	Lịch sử đơn của khách	FR-08
S9	Đăng ký, đăng nhập/đăng xuất bằng JWT	FR-09, FR-10
S10	Phân quyền Khách hàng / Nhân viên / Quản lý	FR-11
S11	Nhân viên cập nhật trạng thái đơn	FR-12
S12	PWA mobile-first, health check, Docker Compose, CI/CD, production HTTPS	NFR-09, 11, 14–17

Bảng 8: Phạm vi mở rộng (Should/Could)

#	Hạng mục	Ưu tiên	Yêu cầu
E1	Hủy đơn chưa thanh toán, tự hủy đơn quá hạn 15 phút, hoàn lại tồn kho	Should	FR-13
E2	Áp mã khuyến mãi (Promotion)	Should	FR-14
E3	Tích điểm thành viên (LoyaltyTransaction)	Should	FR-15
E4	Quản lý sản phẩm và tồn kho cho Quản lý	Should	FR-16
E5	Tìm kiếm, lọc menu	Should	FR-17
E6	Dùng điểm để giảm giá	Could	FR-18
E7	Báo cáo doanh thu theo ngày	Could	FR-19
E8	Thông báo đẩy (Web Push) khi đơn sẵn sàng	Could	FR-20

Bảng 9: Ngoài phạm vi (out-of-scope, Won't)

#	Nội dung	Lý do
O1	Cổng thanh toán thật (VNPay, MoMo, Stripe...)	Đề bài yêu cầu mô phỏng; cần hợp đồng và tài khoản merchant (FR-21)
O2	Giao hàng tận nơi	Ngoài nghiệp vụ quán mang đi (FR-22)
O3	Nhiều chi nhánh	Quán chỉ có một cửa hàng (FR-23)
O4	Đăng nhập Google/Facebook	Không cần cho MVP, tăng rủi ro cấu hình OAuth (FR-24)
O5	Ứng dụng native iOS/Android	Đã có PWA thay thế
O6	Quản lý nguyên liệu	Tồn kho chỉ tính theo số ly của từng sản phẩm
O7	Hoàn tiền tự động	Đơn chỉ hủy được khi chưa thanh toán
O8	Hạ tầng đa vùng, sẵn sàng cao	Vượt nguồn lực của đồ án (NFR-20)

2.1.4 Giả định và ràng buộc

Bảng 10: Giả định

Mã	Giả định
A1	Quán có một cửa hàng; mọi đơn là mang đi hoặc dùng tại quán, khách tự đến quầy nhận.
A2	Khách có smartphone, trình duyệt hiện đại (Chrome Android, Safari iOS) và có Internet khi đặt đơn.
A3	Tiền tệ là VND, số nguyên, giá đã gồm VAT.
A4	Tồn kho tính theo sản phẩm (số ly còn bán được). Size và topping không có tồn kho riêng.
A5	Đơn vị thanh toán là dịch vụ mô phỏng, trả kết quả theo bộ dữ liệu test cố định. Không có tiền thật.
A6	Khách phải đăng nhập mới đặt đơn được. Khách vãng lai chỉ xem menu, chi tiết sản phẩm và giỏ hàng.
A7	Tài khoản Nhân viên và Quản lý được tạo sẵn bằng seed hoặc do Quản lý tạo.
A8	Đơn chưa thanh toán giữ tồn kho tối đa 15 phút, sau đó tự hủy và trả lại tồn kho.
A9	Nhóm dùng gói miễn phí của các nền tảng cloud; cold start vài giây ở lần gọi đầu là chấp nhận được.

Bảng 11: Ràng buộc

Mã	Loại	Ràng buộc
C1	Công nghệ	Bắt buộc Next.js (mobile-first/PWA), NestJS và PostgreSQL. Được bổ sung nhưng không được thay thế.
C2	Công nghệ	Đóng gói bằng Docker; `docker-compose.yml` chạy web, API và DB ở local.
C3	Quy trình	5 task làm tuần tự; tạo phẩm mỗi task phải có trên repo với lịch sử commit/PR rõ ràng.
C4	Quy trình	Mọi thay đổi yêu cầu, sơ đồ hoặc kiến trúc phải đi qua commit/PR và ghi lý do.
C5	Quy trình	Pull request không được merge nếu lint, test hoặc build thất bại.
C6	Cấu trúc	Repo theo cấu trúc `apps/web`, `apps/api`, `docs/`, `tests/`, `.github/workflows/`.
C7	Triển khai	Production có HTTPS; database không public; secret không nằm trong repo hay log CI.
C8	Thời gian	Hai Sprint trong học kỳ I 2026–2027; production phải hoạt động lúc chấm.
C9	Chi phí	0 đồng: chỉ dùng gói miễn phí (GitHub, GHCR, Vercel/Render/Neon hoặc tương đương).
C10	Pháp lý	Không lưu số thẻ đầy đủ hay CVV, kể cả dữ liệu mô phỏng; chỉ lưu 4 số cuối.
C11	Nhân lực	Đề tài do một sinh viên thực hiện, kiêm toàn bộ vai trò; không có người review chéo trong nhóm.

2.1.5 Rủi ro ban đầu

Thang đo xác suất (X) và tác động (T) từ 1 (thấp) đến 3 (cao); Điểm = X × T.

Bảng 12: Rủi ro ban đầu

Mã	Rủi ro	X	T	Điểm	Biện pháp giảm thiểu
R1	Nền tảng miễn phí hết hạn hoặc ngủ đúng lúc chấm	2	3	**6**	Dùng Postgres không hết hạn (Neon); smoke test định kỳ; quy trình redeploy và rollback sẵn
R2	Bán vượt tồn kho khi nhiều khách đặt cùng lúc	2	3	**6**	Trừ kho có điều kiện `stock >= qty` trong transaction; test đồng thời (NFR-13)
R3	Trừ tiền hai lần khi khách bấm lại hoặc mạng chập chờn	2	3	**6**	Idempotency key cho mỗi yêu cầu thanh toán; tối đa 1 payment thành công mỗi đơn (FR-06)
R4	Phạm vi phình to làm trễ luồng chính	3	2	**6**	Ưu tiên MoSCoW; làm Must trước, Should/Could sau khi luồng end-to-end chạy
R5	Chưa quen NestJS, Docker, CI/CD	2	2	4	Dựng khung và CI từ Sprint 1; làm theo tài liệu chính thức; ghi lại trong README
R6	Lộ secret lên GitHub	1	3	3	`.env` trong `.gitignore`; chỉ commit `.env.example`; GitHub Secrets; quét secret trong CI
R7	Tài liệu và code lệch nhau	2	2	4	Ma trận truy vết; mọi thay đổi yêu cầu qua PR
R8	Cold start làm smoke test hoặc demo chậm	2	1	2	Smoke test có retry; "đánh thức" dịch vụ trước demo
R9	Một người làm toàn bộ: quá tải, không ai review chéo	2	3	**6**	Cắt phạm vi theo MoSCoW; CI tự động thay review chéo; mọi thay đổi qua PR có checklist

2.2 Task 1.2 – Yêu cầu chức năng và phi chức năng (SRS)

2.2.1 Quy ước

Mỗi yêu cầu có mã định danh duy nhất (`FR-xx`, `NFR-xx`), mức ưu tiên MoSCoW và tiêu chí chấp nhận kiểm chứng được, đánh mã `AC-xx.y`. Test ở các task sau tham chiếu trực tiếp tới mã AC.

Bảng 13: Quy ước MoSCoW

Mức	Ý nghĩa trong dự án
Must	Bắt buộc cho MVP; thiếu thì luồng end-to-end không chạy hoặc không đạt yêu cầu đề bài.
Should	Quan trọng, làm trong Sprint 2 sau khi các Must đã chạy end-to-end.
Could	Làm nếu còn thời gian; không ảnh hưởng nghiệm thu.
Won't	Đã thống nhất không làm trong phiên bản này; ghi lại để tránh phình phạm vi.

Quy ước chung: tiền tệ VND số nguyên (hiển thị `45.000 đ`); mọi API có tiền tố `/api/v1`; lỗi API trả JSON thống nhất `{ statusCode, code, message, details }`; thời gian lưu UTC, hiển thị theo `Asia/Ho_Chi_Minh`.

2.2.2 Tác nhân và vai trò

Bảng 14: Tác nhân

Tác nhân	Vai trò	Mô tả	Stakeholder
Khách vãng lai	–	Chưa đăng nhập; xem menu, chi tiết sản phẩm, giỏ hàng	SH-01
Khách hàng	`CUSTOMER`	Kế thừa Khách vãng lai; đặt đơn, thanh toán, xem đơn của mình, tích điểm	SH-01
Nhân viên quầy	`STAFF`	Xem đơn đã thanh toán, cập nhật trạng thái chế biến	SH-02
Quản lý cửa hàng	`MANAGER`	Kế thừa Nhân viên; quản lý sản phẩm, tồn kho, khuyến mãi, báo cáo	SH-03
Payment Service	Hệ thống ngoài	Dịch vụ thanh toán mô phỏng xử lý giao dịch Ví/Thẻ	SH-06
Bộ hẹn giờ	Hệ thống nội bộ	Tác vụ định kỳ hủy đơn quá hạn thanh toán	–

2.2.3 Quy tắc nghiệp vụ

Bảng 15: Quy tắc nghiệp vụ

Mã	Quy tắc
BR-01	Mỗi sản phẩm có giá cơ bản (`basePrice`), danh sách size kèm phụ thu (mặc định S +0, M +5.000, L +10.000; cấu hình được theo sản phẩm) và danh sách topping kèm giá.
BR-02	Thành tiền một dòng = (`basePrice` + phụ thu size + Σ giá topping) $\times$ số lượng.
BR-03	Mỗi dòng chọn đúng 1 size và 0–3 topping khác nhau; số lượng mỗi dòng 1–10; một đơn tối đa 20 ly; ghi chú mỗi dòng $\leq$ 100 ký tự.
BR-04	Tạm tính = Σ thành tiền các dòng. Giảm giá = khuyến mãi (BR-13) + đổi điểm (BR-15). Tổng = Tạm tính − Giảm giá, luôn ≥ 0.
BR-05	Server luôn tự tính lại giá từ database, bỏ qua giá client gửi. Lệch `expectedTotal` thì trả 409 `PRICE_CHANGED` kèm giá mới.

Mã	Quy tắc
BR-06	Khi tạo đơn, tên và đơn giá của sản phẩm, size, topping được chụp lại (snapshot) vào `OrderItem`; sửa menu về sau không đổi đơn cũ.
BR-07	Tồn kho được kiểm tra và trừ trong cùng transaction với việc tạo đơn. Một món không đủ thì cả đơn bị từ chối (409 `OUT_OF_STOCK`) và không món nào bị trừ. Tồn kho không bao giờ âm.
BR-08	Đơn `PENDING_PAYMENT` quá 15 phút (`expiresAt`) bị hủy tự động và hoàn lại tồn kho.
BR-09	Mỗi đơn được thanh toán thất bại tối đa 3 lần; lần thứ 3 làm đơn chuyển `CANCELLED` và hoàn kho.
BR-10	Mỗi đơn có tối đa 1 payment `SUCCEEDED`; số tiền thanh toán luôn bằng `Order.total`.
BR-11	Mỗi yêu cầu thanh toán kèm header `Idempotency-Key` (UUID v4), hiệu lực 24 giờ trong phạm vi một người dùng. Cùng khóa, cùng nội dung: trả đúng kết quả cũ và không gọi Payment Service lần hai. Cùng khóa, khác nội dung: 422 `IDEMPOTENCY_KEY_REUSED`.
BR-12	Luồng trạng thái hợp lệ: `PENDING_PAYMENT` → `PAID`; `PENDING_PAYMENT` → `CANCELLED`; `PAID` → `PREPARING` → `READY` → `COMPLETED`. Mọi chuyển khác bị từ chối; không hủy được đơn đã `PAID`.
BR-13	Mã khuyến mãi hợp lệ khi đang bật, trong thời hạn, `usedCount < usageLimit` và Tạm tính ≥ `minOrderValue`. Loại `PERCENT` có trần `maxDiscount`; loại `FIXED` không vượt Tạm tính. Mỗi đơn tối đa 1 mã; `usedCount` chỉ tăng khi đơn `PAID`.
BR-14	Khi đơn chuyển `PAID`, khách được `floor(Tổng / 10.000)` điểm; mỗi đơn chỉ cộng điểm một lần.
BR-15	Đổi điểm (Could): 1 điểm = 1.000 đ, tối đa 50% Tạm tính sau khuyến mãi; điểm bị trừ khi đơn `PAID`.
BR-16	Mã đơn dạng `BL-YYMMDD-NNNN`; `NNNN` tăng dần trong ngày và là số gọi món ở quầy.
BR-17	Khách chỉ xem và thao tác được đơn của chính mình; truy cập đơn người khác trả 404.
BR-18	Không lưu số thẻ đầy đủ và CVV; database chỉ lưu `cardLast4` và `cardBrand`.

Bảng 16: Dữ liệu test của Payment Service mô phỏng

Phương thức	Dữ liệu test	Kết quả	Mã lỗi
Thẻ	`4242 4242 4242 4242`	Thành công	–
Thẻ	`4000 0000 0000 0002`	Thất bại	`CARD_DECLINED`
Thẻ	`4000 0000 0000 9995`	Thất bại	`INSUFFICIENT_FUNDS`
Thẻ	`4000 0000 0000 0119`	Lần đầu chậm hơn 10 giây (timeout); gửi lại cùng khóa thì thành công	`PAYMENT_TIMEOUT`
Thẻ	Số thẻ hợp lệ Luhn khác	Thành công	–
Ví	SĐT `0900000001`	Thất bại	`INSUFFICIENT_FUNDS`
Ví	SĐT `0900000002`	Thất bại	`WALLET_LOCKED`
Ví	SĐT di động hợp lệ khác	Thành công	–

2.2.4 Yêu cầu chức năng

Bảng 17: Tổng hợp yêu cầu chức năng

Mã	Nhóm	Tên yêu cầu	Ưu tiên	Sprint	UC
FR-01	Menu	Xem menu	Must	1	UC-02
FR-02	Chi tiết sản phẩm	Xem chi tiết, chọn size và topping	Must	1/2	UC-02

Mã	Nhóm	Tên yêu cầu	Ưu tiên	Sprint	UC
FR-03	Giỏ hàng	Quản lý giỏ hàng, tính tổng tiền	Must	2	UC-03
FR-04	Đặt đơn	Tạo đơn, kiểm tra và trừ tồn kho	Must	2	UC-01
FR-05	Thanh toán	Thanh toán Ví/Thẻ mô phỏng	Must	2	UC-01, 14
FR-06	Thanh toán	Idempotency khi gửi lại yêu cầu thanh toán	Must	2	UC-01
FR-07	Trạng thái đơn	Xác nhận và theo dõi trạng thái đơn	Must	2	UC-06
FR-08	Lịch sử đơn	Xem lịch sử đơn	Must	2	UC-07
FR-09	Xác thực	Đăng ký tài khoản khách hàng	Must	2	UC-04
FR-10	Xác thực	Đăng nhập, đăng xuất bằng JWT	Must	2	UC-05
FR-11	Xác thực	Phân quyền theo vai trò	Must	2	Tất cả
FR-12	Quản lý trạng thái	Nhân viên cập nhật trạng thái đơn	Must	2	UC-10
FR-13	Đặt đơn	Hủy đơn chưa thanh toán, tự hủy đơn quá hạn	Should	2	UC-12, 15
FR-14	Khuyến mãi	Áp dụng mã khuyến mãi	Should	2	UC-08
FR-15	Loyalty	Tích điểm thành viên	Should	2	UC-09
FR-16	Quản trị	Quản lý sản phẩm và tồn kho	Should	2	UC-11
FR-17	Menu	Tìm kiếm và lọc menu	Should	2	UC-02
FR-18	Loyalty	Đổi điểm lấy giảm giá	Could	–	UC-09
FR-19	Quản trị	Báo cáo doanh thu theo ngày	Could	–	UC-13
FR-20	Thông báo	Thông báo đẩy khi đơn sẵn sàng	Could	–	UC-06
FR-21	Thanh toán	Tích hợp cổng thanh toán thật	Won't	–	–
FR-22	Đặt đơn	Giao hàng tận nơi	Won't	–	–
FR-23	Quản trị	Quản lý nhiều chi nhánh	Won't	–	–
FR-24	Xác thực	Đăng nhập bằng mạng xã hội	Won't	–	–

2.2.4.1 Tiêu chí chấp nhận chi tiết

FR-01 – Xem menu (Must)

AC-01.1 `GET /products` trả 200, không cần token, mảng sản phẩm `isActive = true`; mỗi phần tử có `id`, `name`, `imageUrl`, `basePrice`, `category`, `inStock` (= `stock > 0`).

AC-01.2 Không trả sản phẩm `isActive = false`.

AC-01.3 Màn menu hiện skeleton khi tải; "Chưa có món nào" khi rỗng; thông báo lỗi và nút "Thử lại" khi API lỗi hoặc mất mạng.

AC-01.4 Sản phẩm `inStock = false` hiện nhãn "Hết hàng" và không có nút thêm vào giỏ.

AC-01.5 Ở chiều rộng 360 px trang không cuộn ngang, mỗi sản phẩm hiện đủ ảnh, tên, giá.

FR-02 – Xem chi tiết sản phẩm, chọn size và topping (Must)

AC-02.1 `GET /products/:id` trả 200 với thông tin FR-01, thêm `description`, `sizes[]`, `toppings[]`.

AC-02.2 `id` không tồn tại hoặc sản phẩm ngừng bán: 404 `PRODUCT_NOT_FOUND`; `id` sai định dạng UUID: 400.

AC-02.3 Màn chi tiết chọn sẵn size đầu tiên; chỉ chọn được đúng 1 size.

AC-02.4 Chọn được 0–3 topping; đủ 3 thì các topping còn lại bị vô hiệu hóa.

AC-02.5 Giá cập nhật ngay khi đổi size, topping, số lượng, đúng BR-02. Ví dụ: basePrice 35.000, size L (+10.000), trân châu 5.000, số lượng 2 → 100.000 đ.

AC-02.6 Số lượng chỉ 1–10; ghi chú quá 100 ký tự thì báo lỗi và vô hiệu nút "Thêm vào giỏ".

FR-03 – Quản lý giỏ hàng và tính tổng tiền (Must)

AC-03.1 "Thêm vào giỏ" làm giỏ tăng đúng số lượng, biểu tượng giỏ cập nhật số ly.

AC-03.2 Cùng sản phẩm, size, bộ topping và ghi chú thì gộp một dòng; khác bất kỳ yếu tố nào thì tạo dòng mới.

AC-03.3 Sửa số lượng (1–10), xóa từng dòng; thành tiền và Tạm tính cập nhật đúng BR-02, BR-04.

AC-03.4 Giỏ giữ nguyên sau khi tải lại trang và sau khi đăng nhập.

AC-03.5 Khách vãng lai dùng được giỏ hàng.

AC-03.6 Giỏ rỗng hiện "Giỏ hàng trống", nút "Đặt hàng" bị vô hiệu hóa.

AC-03.7 Thao tác làm tổng số ly vượt 20 thì không thực hiện và hiện thông báo.

FR-04 – Tạo đơn, kiểm tra và trừ tồn kho (Must)

AC-04.1 `POST /orders` không có JWT hợp lệ: 401; giao diện chuyển đăng nhập, giữ giỏ, quay lại thanh toán sau khi đăng nhập.

AC-04.2 Body sai định dạng: 400 kèm lỗi từng trường; không tạo đơn.

AC-04.3 Server tự tính lại giá (BR-05); lệch `expectedTotal`: 409 `PRICE_CHANGED`.

AC-04.4 Thành công: 201 với `id`, `code`, `status = PENDING_PAYMENT`, `subtotal`, `discount`, `total`, `expiresAt`; tồn kho giảm đúng số lượng đặt.

AC-04.5 Một món không đủ tồn kho: 409 `OUT_OF_STOCK` kèm `productIds`; tồn kho mọi sản phẩm giữ nguyên, không có đơn.

AC-04.6 Đặt món đã ngừng bán: 409 `PRODUCT_UNAVAILABLE`.

AC-04.7 Đơn lưu snapshot (BR-06); sửa giá sau khi đặt không đổi `total` của đơn.

AC-04.8 Tạo đơn thành công thì giỏ được xóa và giao diện chuyển sang bước thanh toán.

FR-05 – Thanh toán Ví/Thẻ mô phỏng (Must)

AC-05.1 `POST /payments {orderId, method, card?, wallet?}` yêu cầu JWT; chỉ chủ đơn gọi được, người khác nhận 404.

AC-05.2 FE và BE cùng kiểm tra: số thẻ đúng Luhn, 16 chữ số; hạn chưa qua; CVV 3 chữ số; SĐT ví 10 chữ số. Sai: 400 và không gọi Payment Service.

AC-05.3 Đơn không ở `PENDING_PAYMENT` hoặc đã quá hạn: 409 `ORDER_NOT_PAYABLE`.

AC-05.4 Thành công: 201 `{paymentId, status: SUCCEEDED, orderStatus: PAID}`; `Payment.amount = Order.total`.

AC-05.5 Thất bại: 402 `PAYMENT_FAILED` kèm lý do; Payment `FAILED`, đơn vẫn `PENDING_PAYMENT`; giao diện cho thử lại hoặc đổi phương thức.

AC-05.6 Thất bại lần thứ 3: đơn `CANCELLED`, tồn kho được hoàn (BR-09).

AC-05.7 Payment Service không phản hồi trong 10 giây: 202 `{status: PENDING}`; giao diện tự gửi lại cùng `Idempotency-Key`.

AC-05.8 Bảng `payments` và log không chứa số thẻ đầy đủ hay CVV; chỉ có `cardLast4`, `cardBrand`.

FR-06 – Idempotency khi gửi lại yêu cầu thanh toán (Must)

AC-06.1 Thiếu header `Idempotency-Key` hoặc không phải UUID: 400.

AC-06.2 Cùng khóa, cùng body trong 24 giờ: cùng mã HTTP và body; Payment Service được gọi đúng 1 lần.

AC-06.3 Cùng khóa, khác body: 422 `IDEMPOTENCY_KEY_REUSED`.

AC-06.4 Hai request đồng thời cùng khóa: chỉ một được xử lý, request còn lại nhận cùng kết quả hoặc 409 `REQUEST_IN_PROGRESS`.

AC-06.5 Đơn đã `PAID` nhận khóa mới: 409 `ORDER_ALREADY_PAID`; không bao giờ có 2 payment `SUCCEEDED` cho một đơn.

FR-07 – Xác nhận và theo dõi trạng thái đơn (Must)

AC-07.1 Thanh toán thành công thì chuyển tới `/orders/:id`: mã đơn, số gọi món, món, tổng tiền, phương thức (thẻ hiện 4 số cuối), trạng thái.

AC-07.2 `GET /orders/:id` chỉ trả cho chủ đơn, Staff, Manager; người khác nhận 404.

AC-07.3 Nhân viên đổi trạng thái thì khách thấy trạng thái mới trong ≤ 5 giây, không cần tải lại trang.

AC-07.4 Có dòng thời gian các trạng thái đã qua kèm giờ, phút.

FR-08 – Xem lịch sử đơn (Must)

AC-08.1 `GET /orders?page=1&limit=10` yêu cầu JWT, chỉ trả đơn của người gọi, mới nhất trước, mặc định 10 đơn/trang.

AC-08.2 Mỗi dòng hiện mã đơn, ngày giờ, số ly, tổng tiền, trạng thái; bấm vào mở chi tiết.

AC-08.3 Chưa có đơn: hiện "Bạn chưa có đơn nào" và nút "Xem menu".

FR-09 – Đăng ký tài khoản khách hàng (Must)

AC-09.1 `POST /auth/register` với `email`, `password`, `fullName`, `phone?`: 201; tài khoản vai trò `CUSTOMER`, 0 điểm.

AC-09.2 FE và BE kiểm tra: email hợp lệ; mật khẩu ≥ 8 ký tự có chữ và số; họ tên 2–50 ký tự; SĐT 10 chữ số. Sai: 400.

AC-09.3 Email đã tồn tại (không phân biệt hoa thường): 409 `EMAIL_TAKEN`.

AC-09.4 Mật khẩu lưu hash bcrypt; không response nào chứa mật khẩu hay hash.

FR-10 – Đăng nhập, đăng xuất bằng JWT (Must)

AC-10.1 `POST /auth/login` đúng: 200 `{accessToken, expiresIn: 3600, user}`.

AC-10.2 Sai email hoặc mật khẩu đều trả 401 `INVALID_CREDENTIALS` cùng thông báo.

AC-10.3 Endpoint cần đăng nhập trả 401 khi token thiếu, sai chữ ký hoặc hết hạn (60 phút).

AC-10.4 Đăng xuất xóa token phía client và chuyển về menu.

AC-10.5 Token hết hạn giữa chừng: chuyển đăng nhập kèm `returnUrl`, không mất giỏ hàng.

FR-11 – Phân quyền theo vai trò (Must)

AC-11.1 Ba vai trò `CUSTOMER`, `STAFF`, `MANAGER` nằm trong claim `role` của JWT và được kiểm tra ở server.

AC-11.2 `CUSTOMER` gọi `/staff/*` hoặc `/manager/*`: 403; `STAFF` gọi `/manager/*`: 403.

AC-11.3 `MANAGER` gọi được mọi endpoint của `STAFF`.

AC-11.4 Giao diện ẩn menu không thuộc vai trò; truy cập thẳng URL vẫn bị server chặn.

FR-12 – Nhân viên cập nhật trạng thái đơn (Must)

AC-12.1 `GET /staff/orders` trả đơn `PAID`, `PREPARING`, `READY`, cũ nhất trước; không trả đơn `PENDING_PAYMENT`.

AC-12.2 Màn nhân viên có 3 cột Đã thanh toán – Đang pha – Sẵn sàng; đơn mới xuất hiện trong ≤ 5 giây.

AC-12.3 `PATCH /staff/orders/:id/status` chỉ chấp nhận chuyển đúng BR-12; sai: 409 `INVALID_TRANSITION`.

AC-12.4 Mỗi lần chuyển trạng thái ghi lịch sử: trạng thái cũ, mới, người thực hiện, thời điểm.

FR-13 – Hủy đơn chưa thanh toán và tự hủy đơn quá hạn (Should)

AC-13.1 `POST /orders/:id/cancel` khi đơn `PENDING_PAYMENT`: đơn `CANCELLED` và hoàn kho trong cùng transaction.

AC-13.2 Hủy đơn ở trạng thái khác: 409 `ORDER_NOT_CANCELLABLE`.

AC-13.3 Tác vụ định kỳ chạy mỗi phút hủy đơn quá `expiresAt` và hoàn kho.

FR-14 – Áp dụng mã khuyến mãi (Should)

AC-14.1 `POST /promotions/validate` trả `{valid, discount}` hoặc `{valid: false, reason}`.

AC-14.2 `POST /orders` kèm `promoCode` thì server áp giảm giá theo BR-13; mã không hợp lệ: 422.

AC-14.3 Mã GIAM10 (10%, trần 20.000): Tạm tính 300.000 giảm 20.000; Tạm tính 150.000 giảm 15.000.

AC-14.4 usedCount chỉ tăng khi đơn PAID.

FR-15 – Tích điểm thành viên (Should)

AC-15.1 Đơn chuyển PAID thì tạo LoyaltyTransaction EARN với floor(total/10.000) điểm trong cùng transaction.

AC-15.2 Mỗi đơn tối đa 1 giao dịch EARN; gửi lại thanh toán không cộng điểm lần hai.

AC-15.3 GET /me/loyalty trả số điểm và lịch sử giao dịch điểm.

FR-16 – Quản lý sản phẩm và tồn kho (Should)

AC-16.1 MANAGER tạo, sửa sản phẩm: tên, mô tả, danh mục, giá, size, topping, ảnh, tồn kho, trạng thái bán.

AC-16.2 Tồn kho và giá là số nguyên ≥ 0; giá trị âm: 400.

AC-16.3 Xóa sản phẩm là xóa mềm (isActive = false).

AC-16.4 Thay đổi phản ánh ngay trong GET /products ở lần gọi kế tiếp.

FR-17 – Tìm kiếm và lọc menu (Should)

AC-17.1 GET /products?category=<slug> chỉ trả sản phẩm thuộc danh mục.

AC-17.2 GET /products?q=ca phe tìm được "Cà phê sữa" (không phân biệt hoa thường và dấu).

AC-17.3 Giao diện có tab danh mục, ô tìm kiếm; không có kết quả thì hiện "Không tìm thấy món phù hợp".

FR-18 – Đổi điểm lấy giảm giá (Could)

AC-18.1 Khách chọn số điểm muốn dùng; giảm giá theo BR-15, không vượt số điểm đang có.

AC-18.2 Đơn PAID thì tạo LoyaltyTransaction REDEEM số điểm âm.

FR-19 – Báo cáo doanh thu theo ngày (Could)

AC-19.1 GET /manager/reports/daily trả mỗi ngày: số đơn đã thanh toán, doanh thu, 5 món bán chạy; không tính đơn hủy.

FR-20 – Thông báo đẩy khi đơn sẵn sàng (Could)

AC-20.1 Khách đã cho phép thông báo nhận Web Push trong ≤ 10 giây sau khi đơn chuyển READY.

FR-21 → FR-24 – Không thực hiện (Won't)

FR-21 Không có tích hợp hay khóa API của cổng thanh toán thật; mọi thanh toán qua Payment Service mô phỏng.

FR-22 Không có trường địa chỉ giao hàng hay phí ship.

FR-23 Dữ liệu không có thực thể chi nhánh.

FR-24 Màn đăng nhập chỉ có email và mật khẩu.

2.2.5 Yêu cầu phi chức năng

Bảng 18: Yêu cầu phi chức năng

Mã	Nhóm	Ưu tiên	Yêu cầu và tiêu chí chấp nhận	Kiểm chứng
NFR-01	Hiệu năng	Must	**Phản hồi API:** 20 người dùng đồng thời, GET /products p95 $\leq$ 500 ms; POST /orders, POST /payments p95 $\leq$ 1.000 ms (không tính chờ Payment Service).	k6 trên production
NFR-02	Hiệu năng	Should	**Tải trang:** trang menu mobile (Lighthouse, 4G) Performance $\geq$ 80, LCP $\leq$ 2,5 s, CLS $\leq$ 0,1.	Lighthouse

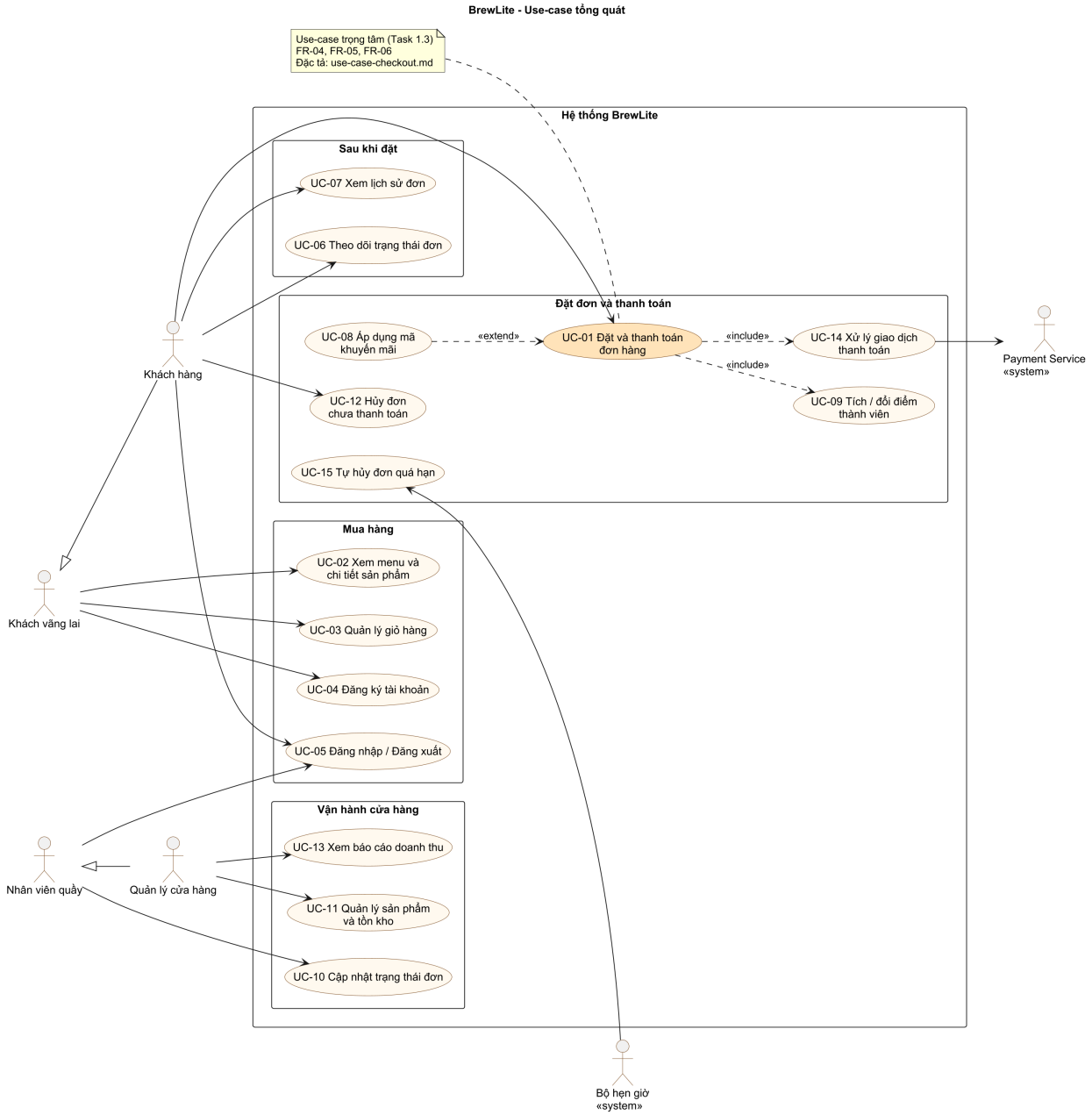
Mã	Nhóm	Ưu tiên	Yêu cầu và tiêu chí chấp nhận	Kiểm chứng
NFR-03	Bảo mật	Must	**Mật khẩu:** bcrypt cost ≥ 10; không API hay log nào trả về mật khẩu, hash.	Unit test, log
NFR-04	Bảo mật	Must	**JWT:** ký HS256 với secret ≥ 32 byte từ biến môi trường; hết hạn 60 phút; token sai hoặc hết hạn bị từ chối.	Integration test
NFR-05	Bảo mật	Must	**Validate:** mọi body, query validate ở backend theo whitelist; dữ liệu sai trả 400, không bao giờ 500; frontend validate cùng quy tắc.	Integration test
NFR-06	Bảo mật	Must	**Secret:** không có secret trong repo hay lịch sử git; chỉ có `.env.example`; quét secret trong CI không phát hiện lỗi; log CI không in secret.	gitleaks trong CI
NFR-07	Bảo mật	Must	**Đường truyền:** frontend và API production chỉ chạy HTTPS; database production không có địa chỉ public.	Cấu hình nền tảng
NFR-08	Bảo mật	Should	**Chống lạm dụng:** đăng nhập giới hạn 5 lần/phút/IP (429); mọi API 100 request/phút/IP; Helmet; CORS chỉ cho domain frontend.	Integration test
NFR-09	Khả dụng	Must	**Mobile-first:** mọi màn hình dùng tốt ở 360–430 px, không cuộn ngang; vùng chạm $\geq 44\times44$ px; từ menu tới thanh toán xong một ly ≤ 8 lần chạm.	Playwright mobile
NFR-10	Khả dụng	Should	**Accessibility:** Lighthouse Accessibility ≥ 90; mọi ô nhập có nhãn; tương phản $\geq 4{,}5{:}1$; dùng được bằng bàn phím.	Lighthouse, axe
NFR-11	Khả dụng	Must	**PWA:** có manifest (icon 192/512, standalone) và service worker; cài đặt được lên màn hình chính; mất mạng có trang offline.	Lighthouse PWA
NFR-12	Khả dụng	Must	**Ngôn ngữ:** giao diện tiếng Việt; tiền dạng 45.000 đ; giờ Việt Nam; thông báo lỗi dễ hiểu, không lộ chi tiết kỹ thuật.	E2E, thủ công
NFR-13	Tin cậy	Must	**Toàn vẹn dữ liệu:** tạo đơn + trừ kho, thanh toán + cập nhật đơn + cộng điểm đều trong transaction. 10 request đồng thời đặt sản phẩm còn 5 ly: đúng 5 thành công, 5 nhận `OUT_OF_STOCK`, kho cuối = 0.	Integration test đồng thời
NFR-14	Tin cậy	Must	**Health check:** `GET /health` trả 200 `{status, db: up}` trong ≤ 1 s; 503 khi mất DB; uptime $\geq$ 99% trong tuần chấm.	Smoke test, ping
NFR-15	Triển khai	Must	**Chạy local:** người ngoài nhóm clone repo, `docker compose up` có web, API, DB trong $\leq$ 10 phút theo README; migration và seed tái tạo được DB.	Thử máy sạch
NFR-16	Triển khai	Must	**CI:** mỗi PR chạy Install $\rightarrow$ Lint $\rightarrow$ Unit Test $\rightarrow$ Build trong ≤ 10 phút; PR không merge được nếu một bước thất bại.	GitHub Actions

Mã	Nhóm	Ưu tiên	Yêu cầu và tiêu chí chấp nhận	Kiểm chứng
NFR-17	Triển khai	Must	**CD, rollback:** push `main` hoặc tag `v*`: lint/test → build → image tag commit SHA → push registry → migrate → deploy → smoke test; smoke test fail thì dừng, rollback về image trước trong ≤ 10 phút.	Pipeline, `deployment.md`
NFR-18	Bảo trì	Must	**Chất lượng mã:** TypeScript strict ở cả web và API; ESLint 0 lỗi; coverage unit test ≥ 70% cho service đơn hàng, thanh toán, tính giá.	Coverage trong CI
NFR-19	Bảo trì	Should	**Logging:** log JSON có `requestId`, method, path, status, thời gian; không ghi mật khẩu, token, số thẻ; không trả stack trace cho client.	Log production
NFR-20	Tin cậy	Won't	Không triển khai đa vùng hay nhiều bản sao DB.	–

Bảng 19: Tổng hợp mức ưu tiên

Mức	Số FR	Số NFR
Must	12 (FR-01 → FR-12)	15
Should	5 (FR-13 → FR-17)	4 (NFR-02, 08, 10, 19)
Could	3 (FR-18 → FR-20)	0
Won't	4 (FR-21 → FR-24)	1 (NFR-20)
Tổng	**24**	**20**

2.2.6 Sơ đồ use-case tổng quát



Hình 1: Sơ đồ use-case tổng quát của hệ thống BrewLite

Bảng 20: Danh sách use-case

Mã	Use-case	Tác nhân chính	Yêu cầu	Ưu tiên
UC-01	**Đặt và thanh toán đơn hàng** (trọng tâm)	Khách hàng, Payment Service	FR-04, 05, 06	Must
UC-02	Xem menu và chi tiết sản phẩm	Khách vãng lai	FR-01, 02, 17	Must
UC-03	Quản lý giỏ hàng	Khách vãng lai	FR-03	Must
UC-04	Đăng ký tài khoản	Khách vãng lai	FR-09	Must

Mã	Use-case	Tác nhân chính	Yêu cầu	Ưu tiên
UC-05	Đăng nhập / Đăng xuất	Khách hàng, Nhân viên, Quản lý	FR-10, 11	Must
UC-06	Theo dõi trạng thái đơn	Khách hàng	FR-07, 20	Must
UC-07	Xem lịch sử đơn	Khách hàng	FR-08	Must
UC-08	Áp dụng mã khuyến mãi (extend UC-01)	Khách hàng	FR-14	Should
UC-09	Tích / đổi điểm thành viên	Khách hàng	FR-15, 18	Should
UC-10	Cập nhật trạng thái đơn	Nhân viên	FR-12	Must
UC-11	Quản lý sản phẩm và tồn kho	Quản lý	FR-16	Should
UC-12	Hủy đơn chưa thanh toán	Khách hàng	FR-13	Should
UC-13	Xem báo cáo doanh thu	Quản lý	FR-19	Could
UC-14	Xử lý giao dịch thanh toán (include của UC-01)	Payment Service	FR-05, 06	Must
UC-15	Tự hủy đơn quá hạn	Bộ hẹn giờ	FR-13	Should

2.2.7 User story

Bảng 21: User story

Mã	User story	Yêu cầu	UC
US-01	Là khách, tôi muốn xem menu theo danh mục và tìm món, để chọn đồ uống nhanh.	FR-01, 17	UC-02
US-02	Là khách, tôi muốn chọn size, topping và thấy giá thay đổi ngay, để biết chính xác số tiền.	FR-02	UC-02
US-03	Là khách, tôi muốn thêm, sửa, xóa món trong giỏ và thấy tổng tiền, để kiểm tra trước khi đặt.	FR-03	UC-03
US-04	Là khách, tôi muốn đăng ký và đăng nhập, để đặt đơn và lưu lịch sử.	FR-09, 10	UC-04, 05
US-05	Là khách hàng, tôi muốn đặt đơn và thanh toán bằng ví hoặc thẻ, để không phải xếp hàng.	FR-04, 05, 06	UC-01
US-06	Là khách hàng, tôi muốn biết khi nào đơn sẵn sàng, để ra quầy lấy đúng lúc.	FR-07	UC-06
US-07	Là khách hàng, tôi muốn xem lại các đơn đã đặt, để đặt lại món quen và đối chiếu chi tiêu.	FR-08	UC-07
US-08	Là khách hàng, tôi muốn nhập mã khuyến mãi, để được giảm giá.	FR-14	UC-08
US-09	Là khách hàng, tôi muốn tích điểm sau mỗi đơn, để được ưu đãi khi quay lại.	FR-15, 18	UC-09
US-10	Là nhân viên quầy, tôi muốn thấy đơn đã thanh toán và đổi trạng thái bằng một chạm, để phục vụ nhanh.	FR-12	UC-10
US-11	Là quản lý, tôi muốn cập nhật sản phẩm, giá và tồn kho, để không bán món đã hết.	FR-16	UC-11
US-12	Là khách hàng, tôi muốn hủy đơn chưa thanh toán, để đổi ý mà không giữ kho của quán.	FR-13	UC-12
US-13	Là quản lý, tôi muốn xem doanh thu theo ngày, để đánh giá kinh doanh.	FR-19	UC-13

2.2.8 Giao diện ngoài

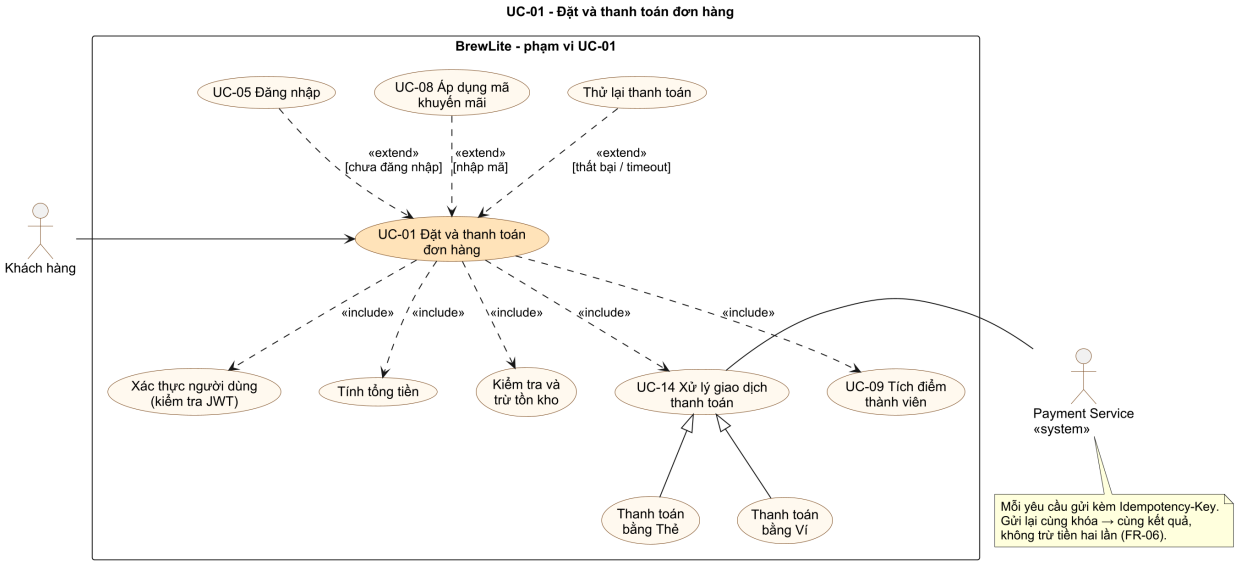
Bảng 22: Danh sách API dự kiến

Method	Endpoint	Quyền	Yêu cầu
GET	`/products`	Công khai	FR-01, 17
GET	`/products/:id`	Công khai	FR-02
POST	`/auth/register`	Công khai	FR-09
POST	`/auth/login`	Công khai	FR-10
GET	`/me`, `/me/loyalty`	CUSTOMER	FR-10, 15
POST	`/orders`	CUSTOMER	FR-04, 14
GET	`/orders`, `/orders/:id`	Chủ đơn, STAFF, MANAGER	FR-07, 08
POST	`/orders/:id/cancel`	Chủ đơn	FR-13
POST	`/payments`	Chủ đơn	FR-05, 06
POST	`/promotions/validate`	CUSTOMER	FR-14
GET	`/staff/orders`	STAFF, MANAGER	FR-12
PATCH	`/staff/orders/:id/status`	STAFF, MANAGER	FR-12
POST, PATCH	`/manager/products[/:id]`	MANAGER	FR-16
GET	`/manager/reports/daily`	MANAGER	FR-19
GET	`/health`	Công khai	NFR-14

Giao diện với Payment Service: HTTPS/JSON đồng bộ, timeout 10 giây. Yêu cầu `POST /charges` `{amount, currency: VND, method, source, reference}` kèm header `Idempotency-Key`; phản hồi `{transactionId, status: SUCCEEDED | FAILED, failureCode?}`. Cùng khóa thì trả cùng kết quả, không tạo giao dịch mới.

2.3 Task 1.3 – Đặc tả use-case trọng tâm: UC-01 Đặt và thanh toán đơn hàng

Lý do chọn: đây là use-case tạo ra doanh thu và đi qua mọi thành phần quan trọng: xác thực, tính giá, tồn kho, transaction, dịch vụ thanh toán bên ngoài, idempotency và loyalty. Nó là phần lõi của luồng bắt buộc *Giỏ hàng → Đặt đơn → Thanh toán → Theo dõi đơn.*



Hình 2: Use-case diagram cho phạm vi UC-01

Bảng 23: Thông tin chung UC-01

Mục	Nội dung
Mã / Tên	UC-01 – Đặt và thanh toán đơn hàng
Mức ưu tiên	Must
Tác nhân chính	Khách hàng (`CUSTOMER`)
Tác nhân phụ	Payment Service (hệ thống thanh toán mô phỏng)
Stakeholder và lợi ích	Khách hàng: đặt nhanh, trả đúng tiền, không bị trừ tiền hai lần. Quản lý: không bán vượt kho, ghi nhận doanh thu chính xác. Nhân viên: chỉ nhận đơn đã thanh toán. Đơn vị thanh toán: yêu cầu hợp lệ, có idempotency key.
Trigger	Khách hàng bấm "Đặt hàng" trên màn Giỏ hàng (`/cart`).
Tiền điều kiện	P1. Giỏ hàng có ít nhất 1 dòng hợp lệ (FR-03). P2. Quán đang hoạt động; API, database sẵn sàng. (Khách chưa đăng nhập vẫn bắt đầu được; xử lý ở luồng A1.)
Hậu điều kiện – thành công	S1. Có `Order PAID` với mã `BL-YYMMDD-NNNN` và snapshot giá. S2. Tồn kho giảm đúng số lượng đặt. S3. Có đúng 1 `Payment SUCCEEDED`, `amount = Order.total`, chỉ lưu 4 số cuối thẻ. S4. Có `LoyaltyTransaction EARN`. S5. Đơn xuất hiện trên bảng đơn nhân viên. S6. Khách thấy màn xác nhận có số gọi món. S7. Giỏ hàng đã được xóa.
Hậu điều kiện – thất bại	F1. Không có payment `SUCCEEDED`, khách không bị trừ tiền. F2. Đơn chưa tạo thì tồn kho giữ nguyên; đơn đã tạo thì ở `PENDING_PAYMENT` (còn thử lại được) hoặc `CANCELLED` và đã hoàn kho. F3. Khách thấy lý do thất bại bằng tiếng Việt.
Yêu cầu liên quan	FR-03, 04, 05, 06, 07, 10, 13, 14, 15; NFR-04, 05, 13; BR-02 → BR-11, BR-13, 14, 17, 18
Tần suất	Cao: khoảng 100–300 đơn/ngày, dồn vào giờ cao điểm

2.3.1 Luồng chính

Bảng 24: Luồng chính UC-01

Bước	Tác nhân	Hành động
1	Khách hàng	Ở màn Giỏ hàng, bấm "Đặt hàng".
2	UI	Kiểm tra khách đã đăng nhập và token còn hạn (chưa đăng nhập → A1).
3	UI	Hiện màn Thanh toán: danh sách món, Tạm tính, ô mã khuyến mãi, chọn phương thức Thẻ hoặc Ví.
4	Khách hàng	Chọn phương thức, nhập thông tin thẻ (số thẻ, hạn, CVV) hoặc ví (SĐT) (nhập mã khuyến mãi → A2).
5	UI	Validate dữ liệu ở client (sai → E1).
6	Khách hàng	Bấm "Thanh toán".
7	UI	Gửi `POST /orders` kèm JWT, `items`, `promoCode?`, `expectedTotal`.
8	API	Xác thực JWT (hết hạn → E2), validate body (sai → E1).
9	API	Tính lại giá từ database theo BR-02, BR-04 (lệch giá → E3).
10	API	Mở transaction: kiểm tra và trừ tồn kho từng sản phẩm (không đủ → E4); tạo `Order` (`PENDING_PAYMENT`, `expiresAt` = now + 15 phút) và các `OrderItem`; commit.
11	API	Trả 201 kèm `orderId`, `code`, `total`, `expiresAt`.
12	UI	Xóa giỏ hàng; sinh `Idempotency-Key` (UUID v4) cho lần thanh toán.
13	UI	Gửi `POST /payments` kèm JWT, `Idempotency-Key`, `orderId`, `method`, thông tin thẻ/ví.
14	API	Kiểm tra idempotency key (đã có → A3); kiểm tra đơn thuộc khách, đang `PENDING_PAYMENT`, chưa quá hạn (không → E5).
15	API	Tạo `Payment` (`PENDING`); gọi Payment Service `POST /charges` với cùng `Idempotency-Key`.
16	Payment Service	Xử lý giao dịch, trả `SUCCEEDED` (thất bại → A4; quá 10 giây → A5).
17	API	Một transaction: `Payment` → `SUCCEEDED`; `Order` → `PAID` và ghi lịch sử trạng thái; tăng `Promotion.usedCount` (nếu có); tạo `LoyaltyTransaction EARN`; lưu response theo idempotency key.
18	API	Trả 201 `{status: SUCCEEDED, orderStatus: PAID}`.
19	UI	Chuyển tới `/orders/:id`: "Thanh toán thành công", mã đơn, số gọi món, tổng tiền, điểm vừa tích.
20	Hệ thống	Đơn xuất hiện trên bảng đơn nhân viên (≤ 5 giây). Kết thúc use-case.

2.3.2 Luồng thay thế

Bảng 25: Luồng thay thế UC-01

Mã	Điều kiện	Xử lý	Quay về
A1	Chưa đăng nhập hoặc token hết hạn (bước 2)	UI chuyển tới `/login?returnUrl=/checkout`, giữ nguyên giỏ; khách đăng nhập (UC-05) hoặc đăng ký (UC-04); thành công thì quay lại `/checkout`. Khách hủy đăng nhập thì kết thúc, giỏ vẫn còn.	Bước 3
A2	Nhập mã khuyến mãi (bước 4), UC-08	UI gọi `POST /promotions/validate`; hợp lệ thì hiện số tiền giảm và Tổng mới; không hợp lệ thì hiện lý do, khách sửa hoặc bỏ mã.	Bước 4

Mã	Điều kiện	Xử lý	Quay về
A3	Client gửi lại `POST /payments` cùng `Idempotency-Key` (bước 14)	API trả đúng response cũ đã lưu, không gọi lại Payment Service. Request trước cùng khóa còn xử lý thì trả 409 `REQUEST_IN_PROGRESS`, UI chờ 2 giây rồi gửi lại.	Bước 18
A4	Payment Service trả `FAILED` (bước 16)	Lưu `Payment FAILED` kèm mã lỗi; đơn vẫn `PENDING_PAYMENT`, kho vẫn giữ; trả 402; UI hiện lý do, nút "Thử lại" và "Đổi phương thức"; lần thử mới dùng khóa mới. Thất bại lần 3 → E6.	Bước 4
A5	Payment Service không phản hồi trong 10 giây (bước 16)	Giữ `Payment PENDING`, trả 202; UI hiện "Đang xử lý" và gửi lại cùng khóa sau 3 giây (tối đa 5 lần); API gọi lại Payment Service cùng khóa và nhận kết quả của giao dịch cũ.	Bước 17 hoặc A4
A6	Thanh toán lại một đơn `PENDING_PAYMENT` có sẵn	UI mở `/checkout?orderId=...`, bỏ qua bước 7–11.	Bước 12

2.3.3 Luồng ngoại lệ

Bảng 26: Luồng ngoại lệ UC-01

Mã	Điều kiện	Xử lý của hệ thống	Kết quả
E1	Dữ liệu không hợp lệ (bước 5, 8)	Client chặn và báo lỗi từng ô; tới server thì trả 400 kèm lỗi từng trường.	Không tạo đơn, không gọi Payment Service; khách sửa và làm lại từ bước 4.
E2	JWT thiếu, sai hoặc hết hạn (bước 8, 14)	Trả 401.	Chuyển sang A1.
E3	Giá đã thay đổi (bước 9)	Trả 409 `PRICE_CHANGED` kèm giá mới.	UI cập nhật giỏ, báo giá thay đổi, quay về bước 3.
E4	**Hết hàng** (bước 10)	Rollback transaction; trả 409 `OUT_OF_STOCK` kèm `productIds`.	Không tạo đơn, không trừ kho; UI đánh dấu "Hết hàng" trên các dòng đó.
E5	Đơn không còn thanh toán được (bước 14)	Trả 409 `ORDER_ALREADY_PAID` hoặc `ORDER_NOT_PAYABLE`.	Đã `PAID`: chuyển màn xác nhận. Đã hủy/hết hạn: báo và gợi ý đặt lại.
E6	Thanh toán thất bại lần thứ 3	Một transaction: `Order CANCELLED`, hoàn kho, ghi lịch sử.	Kết thúc thất bại (F1–F3).
E7	Đơn quá 15 phút chưa thanh toán	Bộ hẹn giờ (UC-15) hủy đơn và hoàn kho.	Kết thúc thất bại.
E8	Lỗi hệ thống (500, mất kết nối DB)	Rollback; log lỗi kèm `requestId`; trả 500/503 với thông báo chung.	UI báo lỗi, giữ dữ liệu đã nhập; gửi lại thanh toán cùng khóa nên an toàn.

2.3.4 Quy tắc đặc biệt

- Số tiền thanh toán luôn lấy từ `Order.total` trên server, không lấy từ client (BR-05, BR-10).
- Mỗi đơn có tối đa 1 payment `SUCCEEDED`, đảm bảo bằng ràng buộc unique có điều kiện trong database.
- Idempotency key hiệu lực 24 giờ, gắn với người dùng (BR-11).
- Không lưu số thẻ đầy đủ và CVV (BR-18).
- Bước 10 và bước 17 đều là transaction; thất bại giữa chừng không để lại dữ liệu nửa vời (NFR-13).

2.3.5 Liên kết use-case – yêu cầu – tiêu chí chấp nhận

Bảng 27: Liên kết UC-01 với yêu cầu

Phần của UC-01	Yêu cầu	Tiêu chí chấp nhận dùng để viết test
Bước 2, A1, E2	FR-10, NFR-04	AC-04.1, AC-10.3, AC-10.5
Bước 3, A2	FR-03, FR-14	AC-03.3, AC-14.1 → AC-14.3
Bước 5, 8, E1	FR-04, FR-05, NFR-05	AC-04.2, AC-05.2
Bước 9, E3	FR-04	AC-04.3
Bước 10–11, E4	FR-04, NFR-13	AC-04.4 → AC-04.7
Bước 12–18	FR-05	AC-05.1, AC-05.3, AC-05.4, AC-05.8
A3, A5, E8	FR-06	AC-05.7, AC-06.1 → AC-06.5
A4, E6	FR-05, FR-13	AC-05.5, AC-05.6
E5, E7	FR-05, FR-13	AC-05.3, AC-06.5, AC-13.3
Bước 17	FR-14, FR-15	AC-14.4, AC-15.1, AC-15.2
Bước 19–20	FR-07, FR-12	AC-07.1, AC-12.2

2.3.6 Kịch bản kiểm thử nghiệm thu

Bảng 28: Kịch bản nghiệm thu rút từ UC-01

Mã	Kịch bản	Dữ liệu	Kết quả mong đợi
UAT-01	Luồng chính bằng thẻ	Thẻ `4242...4242`, 2 ly	Đơn `PAID`, kho −2, 1 payment `SUCCEEDED`, màn xác nhận có số gọi món
UAT-02	Luồng chính bằng ví	SDT `0912345678`	Như UAT-01 với phương thức Ví
UAT-03	Chưa đăng nhập (A1)	Khách vãng lai có giỏ 1 ly	Chuyển đăng nhập, giỏ còn nguyên, quay lại thanh toán
UAT-04	Hết hàng (E4)	Sản phẩm còn 1, đặt 2	409 `OUT_OF_STOCK`, không có đơn, kho vẫn 1
UAT-05	Thẻ bị từ chối (A4)	Thẻ `4000...0002`	402; đơn `PENDING_PAYMENT`; thử lại bằng `4242...` thì thành công
UAT-06	Thất bại 3 lần (E6)	3 lần thẻ `...0002`	Đơn `CANCELLED`, kho được hoàn
UAT-07	Gửi lại cùng khóa (A3)	2 lần cùng `Idempotency-Key`	Cùng response; Payment Service gọi 1 lần; 1 payment
UAT-08	Timeout (A5)	Thẻ `4000...0119`	Lần đầu 202 `PENDING`; gửi lại cùng khóa thì `SUCCEEDED`
UAT-09	Mã khuyến mãi (A2)	`GIAM10`, Tạm tính 150.000	Tổng 135.000; `usedCount` +1 sau khi `PAID`
UAT-10	Đặt đồng thời	10 khách cùng đặt 1 ly, kho còn 5	Đúng 5 đơn thành công, kho = 0

3 Task 2 – Mô hình hóa hệ thống

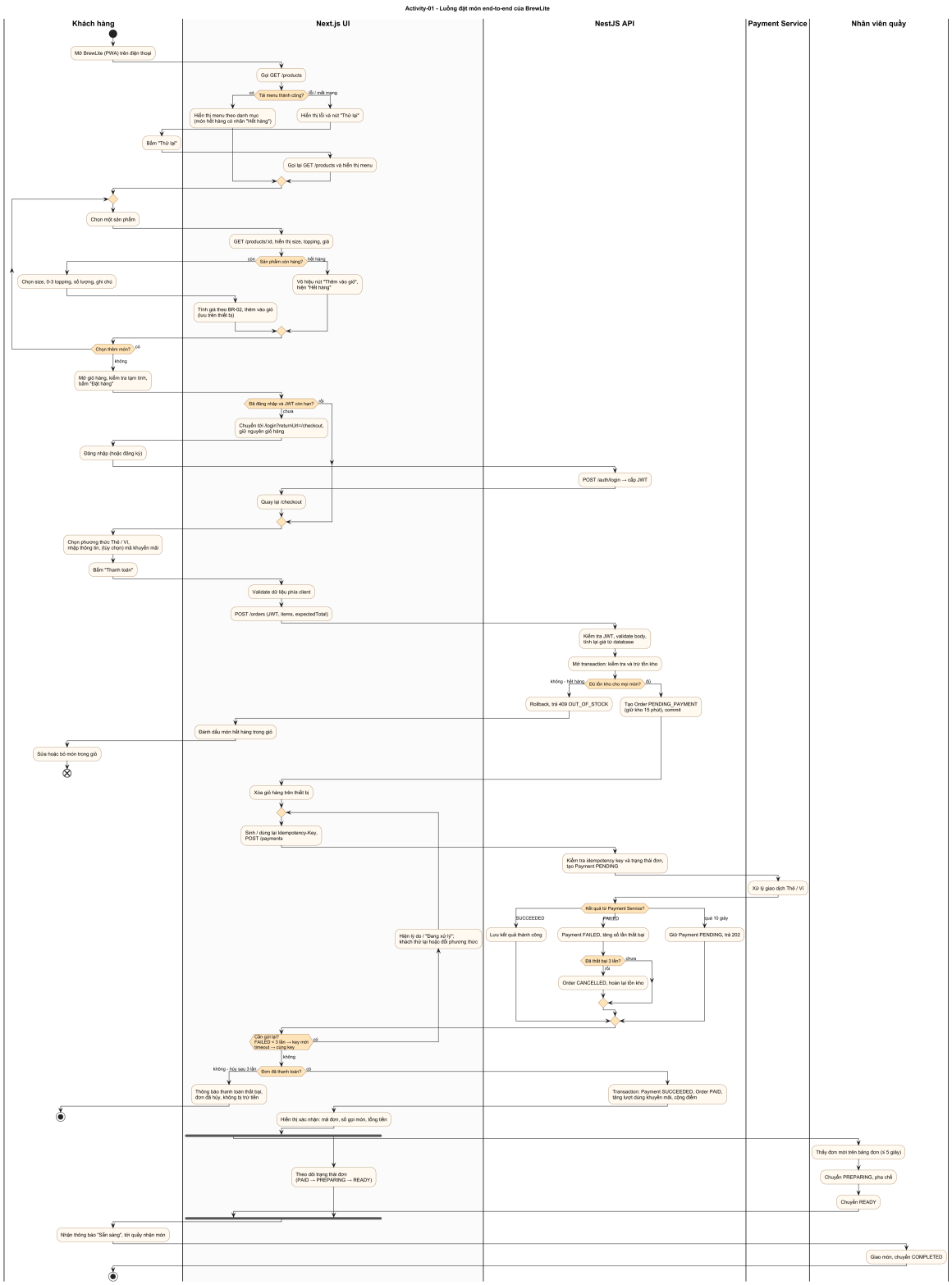
Task 2 mô hình hóa hành vi và cấu trúc của BrewLite từ đặc tả ở Task 1. Mọi sơ đồ được viết bằng PlantUML; tệp nguồn `.puml` và ảnh `.svg` nằm trong `docs/diagrams/`. Mã sơ đồ dùng trong ma trận truy vết:

Bảng 29: Danh sách sơ đồ Task 2

Mã	Task	Nội dung	Tệp nguồn
Activity-01	2.1	Luồng end-to-end: Xem menu → Theo dõi đơn, có swimlane	`activity-order-e2e.puml`
Sequence-01	2.2	UC-01 Đặt và thanh toán đơn hàng	`sequence-checkout.puml`
Class-01	2.3	Mô hình domain	`class-domain.puml`
State-01	bổ trợ	Vòng đời Order và Payment	`state-order-payment.puml`

3.1 Task 2.1 – Activity diagram

Sơ đồ Activity-01 mô tả toàn bộ luồng nghiệp vụ tối thiểu của đề bài (*Xem menu → Chọn tùy chọn → Giỏ hàng → Đặt đơn → Thanh toán → Theo dõi đơn*) qua 5 swimlane: hai làn người dùng (Khách hàng, Nhân viên quầy) và ba làn hệ thống (Next.js UI, NestJS API, Payment Service).



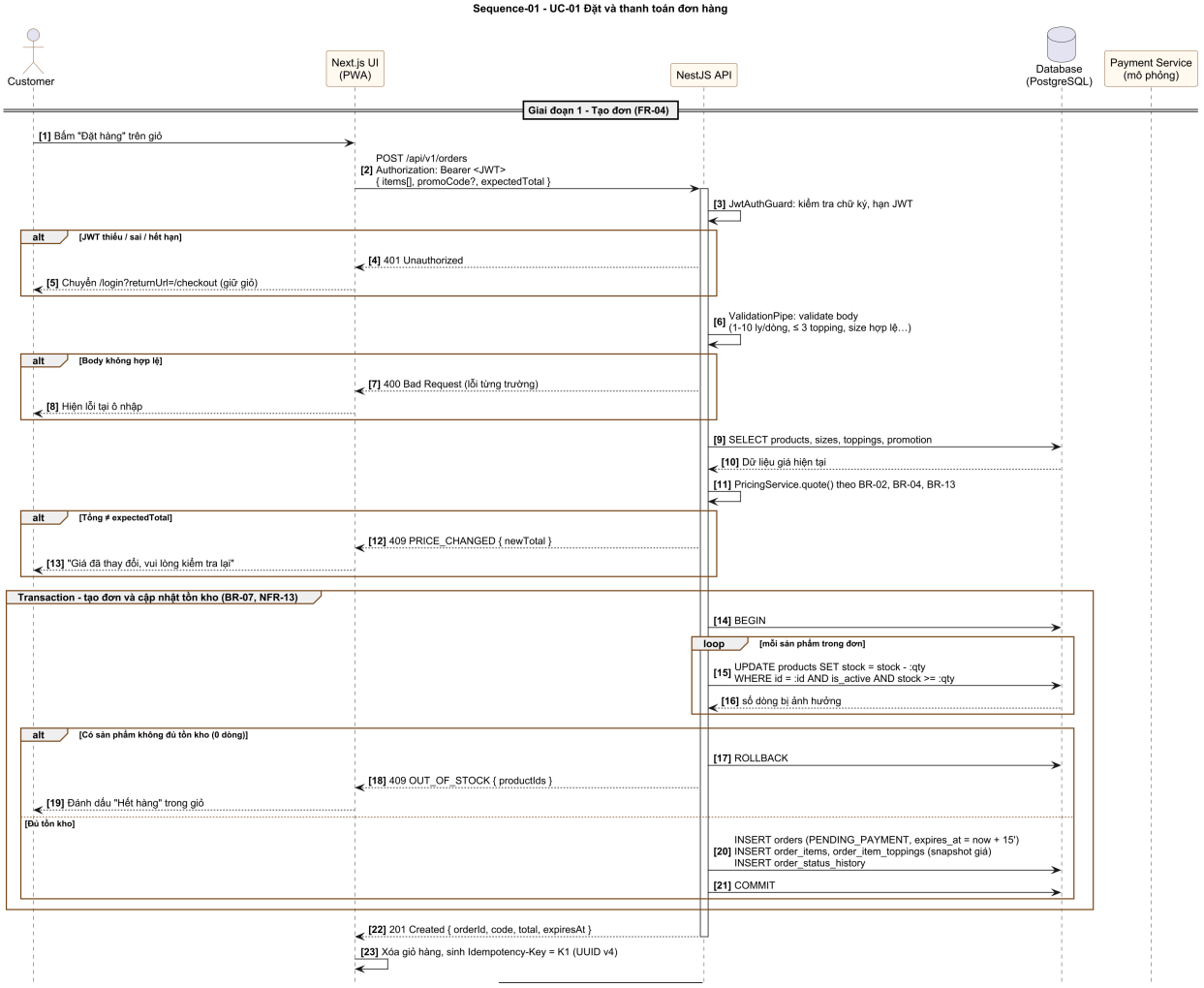
Hình 3: Activity-01 – Luồng đặt món end-to-end (swimlane)

Bảng 30: Activity-01 đối chiếu yêu cầu đề bài

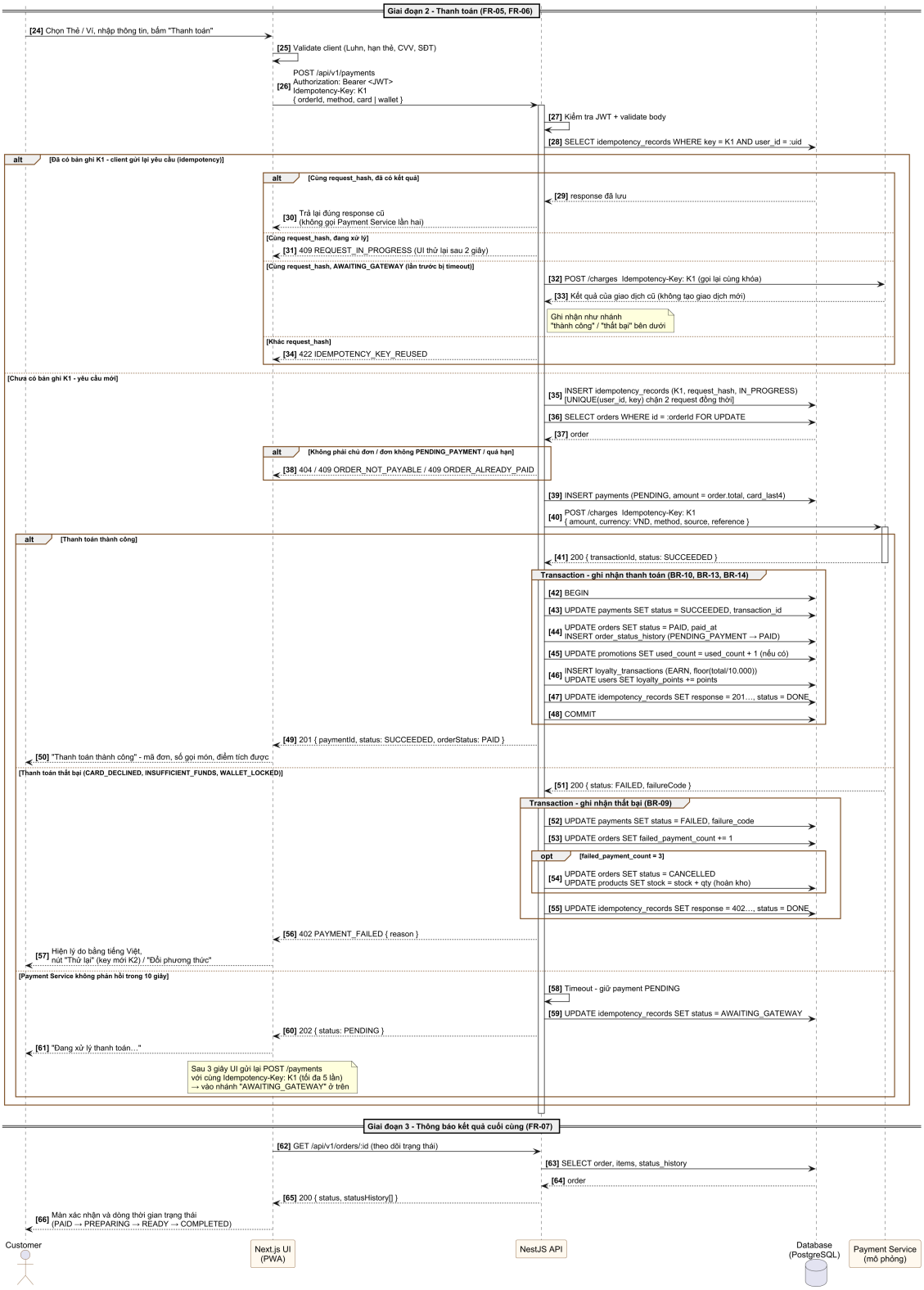
Yêu cầu của đề	**Thể hiện trong sơ đồ**
Điểm bắt đầu, kết thúc	1 điểm bắt đầu; 2 điểm kết thúc (đơn hoàn tất; đơn bị hủy sau 3 lần thất bại); 1 flow-final (dừng vì hết hàng, khách quay lại sửa giỏ).
Hành động người dùng và hệ thống	Làn Khách hàng, Nhân viên quầy (người dùng); Next.js UI, NestJS API, Payment Service (hệ thống).
Decision / merge	Tải menu thành công?; Còn hàng?; Chọn thêm món?; Đã đăng nhập?; Đủ tồn kho?; Kết quả thanh toán (3 nhánh); Đã thất bại 3 lần?; Cần gửi lại?; Đơn đã thanh toán?
Swimlane	5 làn như trên.
Hết hàng	Hai chỗ: ở màn chi tiết sản phẩm (không cho thêm vào giỏ, AC-01.4) và khi tạo đơn (rollback, 409 `OUT_OF_STOCK`, BR-07).
Chưa đăng nhập	Chuyển `/login?returnUrl=/checkout`, giữ giỏ hàng, quay lại thanh toán sau khi đăng nhập (A1, AC-04.1).
Thanh toán thất bại	`FAILED`: thử lại bằng key mới; lần thứ 3 thì hủy đơn và hoàn kho (A4, E6, BR-09). Timeout: gửi lại cùng key (A5).

3.2 Task 2.2 – Sequence diagram

Sequence-01 hiện thực use-case trọng tâm UC-01 với đủ 5 lifeline theo đề: *Customer* → *Next.js UI* → *NestJS API* → *Database* → *Payment Service*. Sơ đồ chia ba giai đoạn: tạo đơn (`POST /orders`), thanh toán (`POST /payments`) và thông báo kết quả cuối cùng (`GET /orders/:id`). Bản SVG đầy đủ độ phân giải nằm tại `docs/diagrams/sequence-checkout.svg`.



Hình 4: Sequence-01 (phần 1/2) – Giai đoạn 1: tạo đơn, kiểm tra JWT, validate, transaction trừ kho



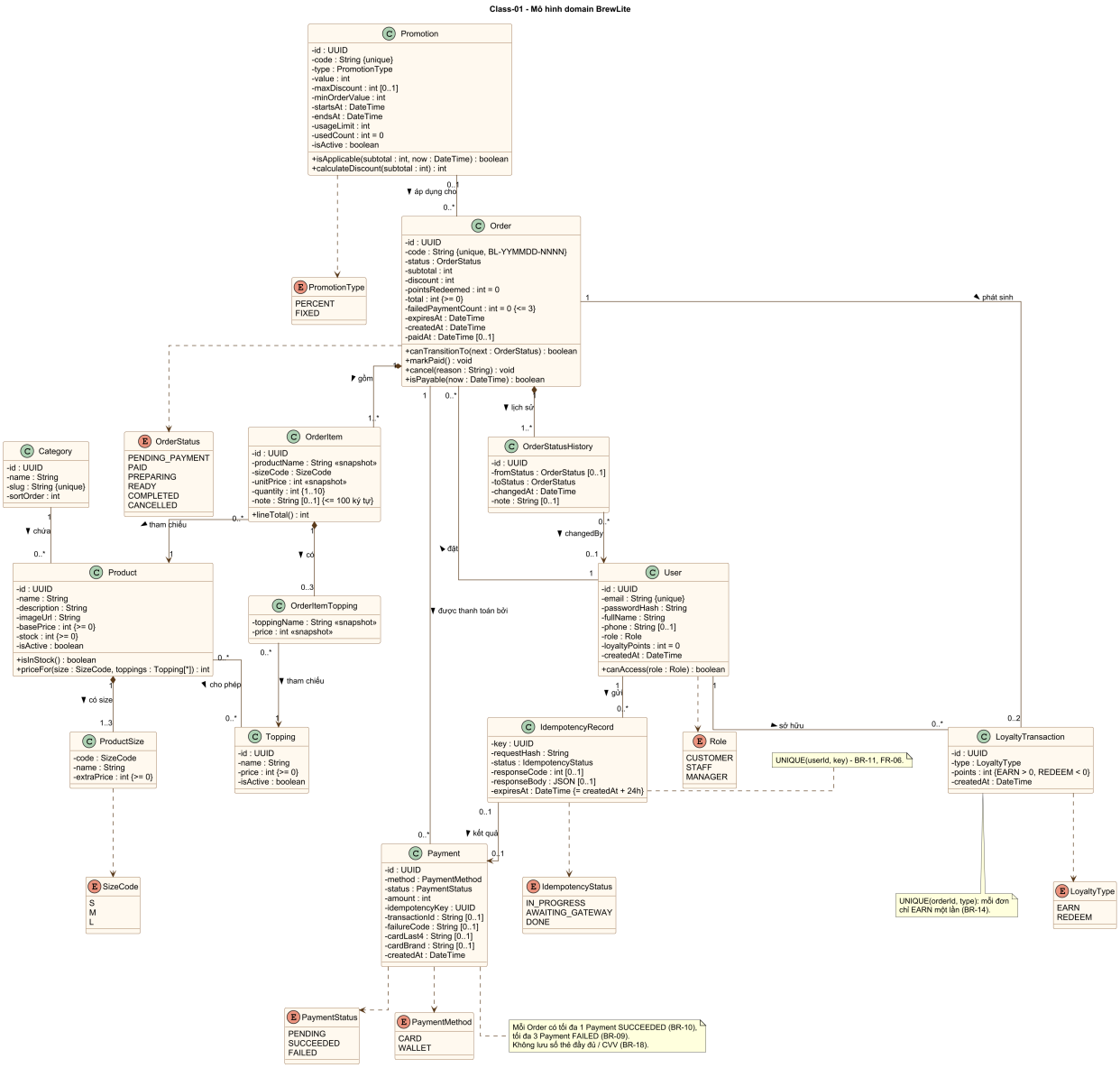
Hình 5: Sequence-01 (phần 2/2) – Giai đoạn 2–3: thanh toán, idempotency, `alt` thành công/thất bại/timeout, thông báo kết quả

Bảng 31: Sequence-01 đối chiếu yêu cầu đề bài

Yêu cầu của đề	**Thể hiện trong sơ đồ**
Request/response, thứ tự gọi API	`POST /orders` (bước 2–22) → `POST /payments` (từ bước 26) → `GET /orders/:id` (giai đoạn 3).
Kiểm tra JWT và validate	`JwtAuthGuard` (bước 3, nhánh 401); `ValidationPipe` (bước 6, nhánh 400); lặp lại ở `POST /payments`.
Transaction tạo đơn và cập nhật tồn kho	Khối *Transaction – tạo đơn và cập nhật tồn kho*: `UPDATE products SET stock = stock - qty WHERE stock >= qty`; không đủ thì `ROLLBACK` và 409.
`alt` thành công / thất bại	`alt` *Thanh toán thành công / thất bại / timeout 10 giây.* Thành công và thất bại đều được ghi trong transaction.
Idempotency khi gửi lại	`alt` *Đã có bản ghi K1*: trả lại response cũ (không gọi Payment Service); 409 khi đang xử lý; 422 khi khác body; gọi lại cổng cùng khóa sau timeout. Bản ghi mới được chèn với ràng buộc `UNIQUE(user_id, key)` để chặn hai request đồng thời.
Thông báo kết quả cuối	"Thanh toán thành công" kèm mã đơn, số gọi món, điểm tích được; hoặc lý do thất bại; giai đoạn 3 hiển thị dòng thời gian trạng thái.

3.3 Task 2.3 – Class diagram

Class-01 là mô hình domain gồm 7 lớp bắt buộc (User, Product, Order, OrderItem, Payment, Promotion, LoyaltyTransaction) và 6 lớp bổ trợ cần để khớp với SRS: Category, ProductSize, Topping, OrderItemTopping, OrderStatusHistory, IdempotencyRecord.



Hình 6: Class-01 – Mô hình domain BrewLite

Bảng 32: Các lớp domain

Lớp	Vai trò và ràng buộc chính	Truy vết
User	Tài khoản; `email` unique; `role`; `loyaltyPoints`	FR-09, FR-10, FR-11
Product	Sản phẩm; `basePrice`, `stock` $\geq$ 0; `isActive` (xóa mềm)	FR-01, FR-02, FR-16, BR-07

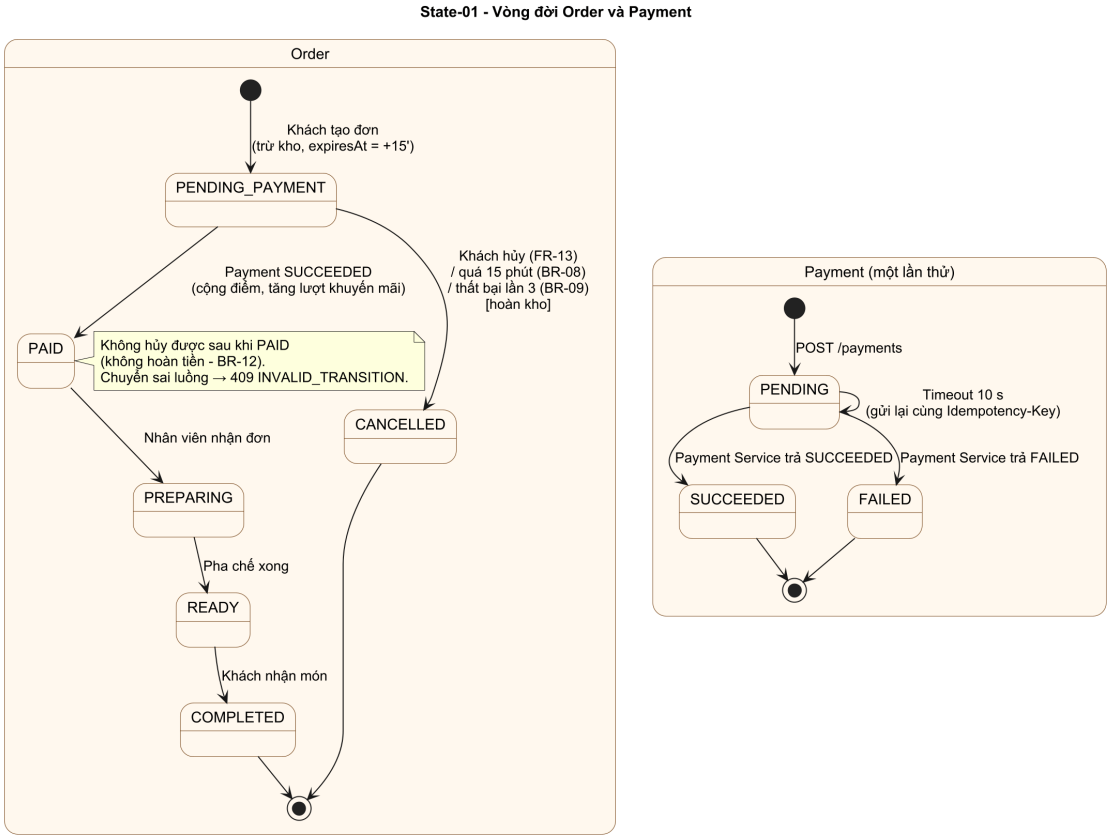
Lớp	Vai trò và ràng buộc chính	Truy vết
Order	Đơn hàng; mã `BL-YYMMDD-NNNN`; `expiresAt`; `failedPaymentCount` $\leq$ 3	FR-04, BR-08, BR-09, BR-16
OrderItem	Dòng đơn; snapshot tên và đơn giá; số lượng 1–10	BR-03, BR-06
Payment	Một lần thử thanh toán; chỉ lưu `cardLast4`; tối đa 1 `SUCCEEDED` mỗi đơn	FR-05, BR-10, BR-18
Promotion	Mã giảm giá `PERCENT`/`FIXED`, trần, thời hạn, lượt dùng	FR-14, BR-13
LoyaltyTransaction	Cộng/trừ điểm; `UNIQUE(orderId, type)`	FR-15, FR-18, BR-14
Category	Danh mục menu, `slug` unique	FR-17
ProductSize	Size S/M/L kèm phụ thu	BR-01
Topping	Topping kèm giá	BR-01, BR-03
OrderItemTopping	Snapshot topping đã chọn (0–3 mỗi dòng)	BR-03, BR-06
OrderStatusHistory	Lịch sử chuyển trạng thái, người thực hiện	AC-07.4, AC-12.4
IdempotencyRecord	Kết quả đã lưu theo khóa; `UNIQUE(userId, key)`; hết hạn 24 giờ	FR-06, BR-11

Bảng 33: Enum trạng thái

Enum	Giá trị	Truy vết
OrderStatus	`PENDING_PAYMENT`, `PAID`, `PREPARING`, `READY`, `COMPLETED`, `CANCELLED`	BR-12
PaymentStatus	`PENDING`, `SUCCEEDED`, `FAILED`	FR-05
PaymentMethod	`CARD`, `WALLET`	FR-05
PromotionType	`PERCENT`, `FIXED`	BR-13
LoyaltyType	`EARN`, `REDEEM`	BR-14, BR-15
Role	`CUSTOMER`, `STAFF`, `MANAGER`	FR-11
SizeCode	`S`, `M`, `L`	BR-01
IdempotencyStatus	`IN_PROGRESS`, `AWAITING_GATEWAY`, `DONE`	FR-06

3.4 Sơ đồ trạng thái bổ trợ

State-01 cố định vòng đời Order và Payment, để ba sơ đồ trên dùng chung một định nghĩa trạng thái.



Hình 7: State-01 – Vòng đời Order và Payment

3.5 Kiểm tra nhất quán với SRS và use-case

Tiêu chí nghiệm thu Task 2: *ba loại sơ đồ không mâu thuẫn với SRS, use-case hoặc mô hình dữ liệu.*

Bảng 34: Đối chiếu nhất quán giữa các sơ đồ và SRS

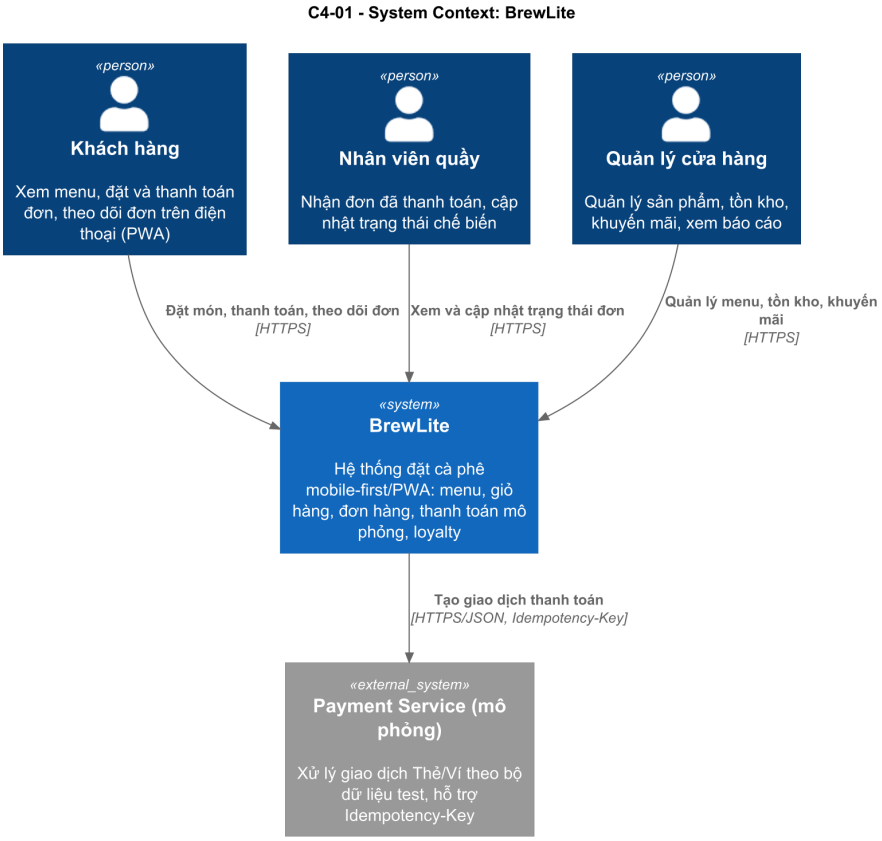
Khái niệm	**SRS / Use-case**	**Activity-01**	**Sequence-01**	**Class-01 / State-01**
Trạng thái đơn	BR-12	Đúng thứ tự, nhân viên đổi trạng thái	`PENDING_PAYMENT` → `PAID`; `CANCELLED` sau 3 lần	`OrderStatus` 6 giá trị; 7 chuyển trạng thái
Trạng thái thanh toán	AC-05.4 → 05.7	Switch 3 nhánh	`alt` 3 nhánh	`PaymentStatus` 3 giá trị
Trừ kho	BR-07	Transaction trừ kho	`UPDATE ... stock >= qty`	`stock` ≥ 0
Giữ kho 15 phút	BR-08	"giữ kho 15 phút"	`expires_at = now + 15'`	`Order.expiresAt`
Tối đa 3 lần thất bại	BR-09	"Đã thất bại 3 lần?"	`opt` count = 3	`failedPaymentCount` ≤ 3
Idempotency	BR-11, FR-06	Sinh / dùng lại key	`alt` đã có K1	`IdempotencyRecord`
Khuyến mãi	BR-13, FR-14	Nhập mã khi thanh toán	`used_count + 1` khi PAID	`Promotion.usedCount`
Tích điểm	BR-14, FR-15	Cộng điểm khi PAID	`INSERT loyalty_transactions`	`UNIQUE(orderId, type)`
Snapshot giá	BR-06	–	snapshot trong `order_items`	«snapshot»
Không lưu số thẻ	BR-18	–	`card_last4`	Chỉ có `cardLast4`
Bắt buộc đăng nhập	A6, AC-04.1	Nhánh chưa đăng nhập	Nhánh 401	`User.role`

4 Task 3 – Thiết kế kiến trúc

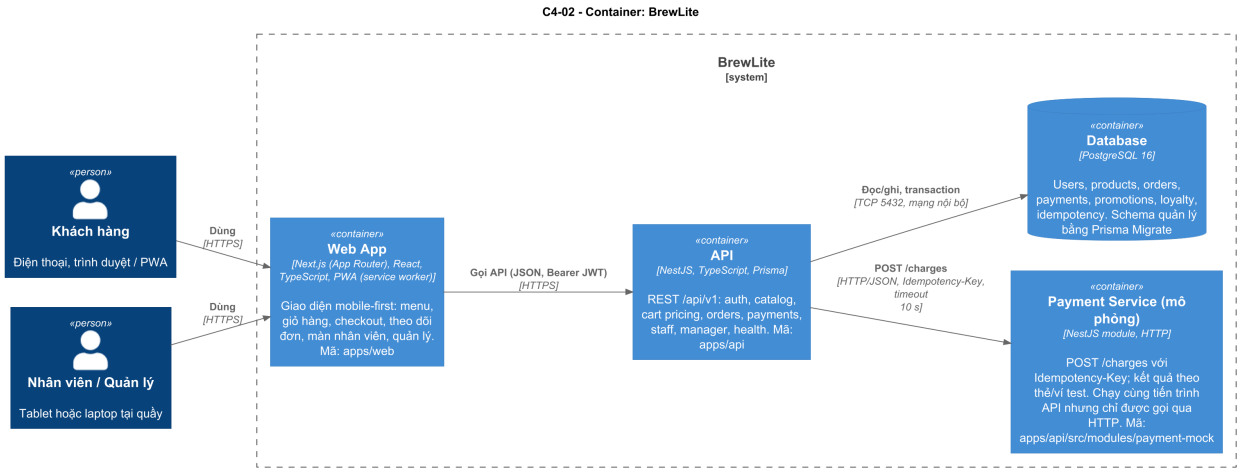
4.1 Task 3.1 – Phương pháp và quyết định kiến trúc

4.1.1 Phương pháp

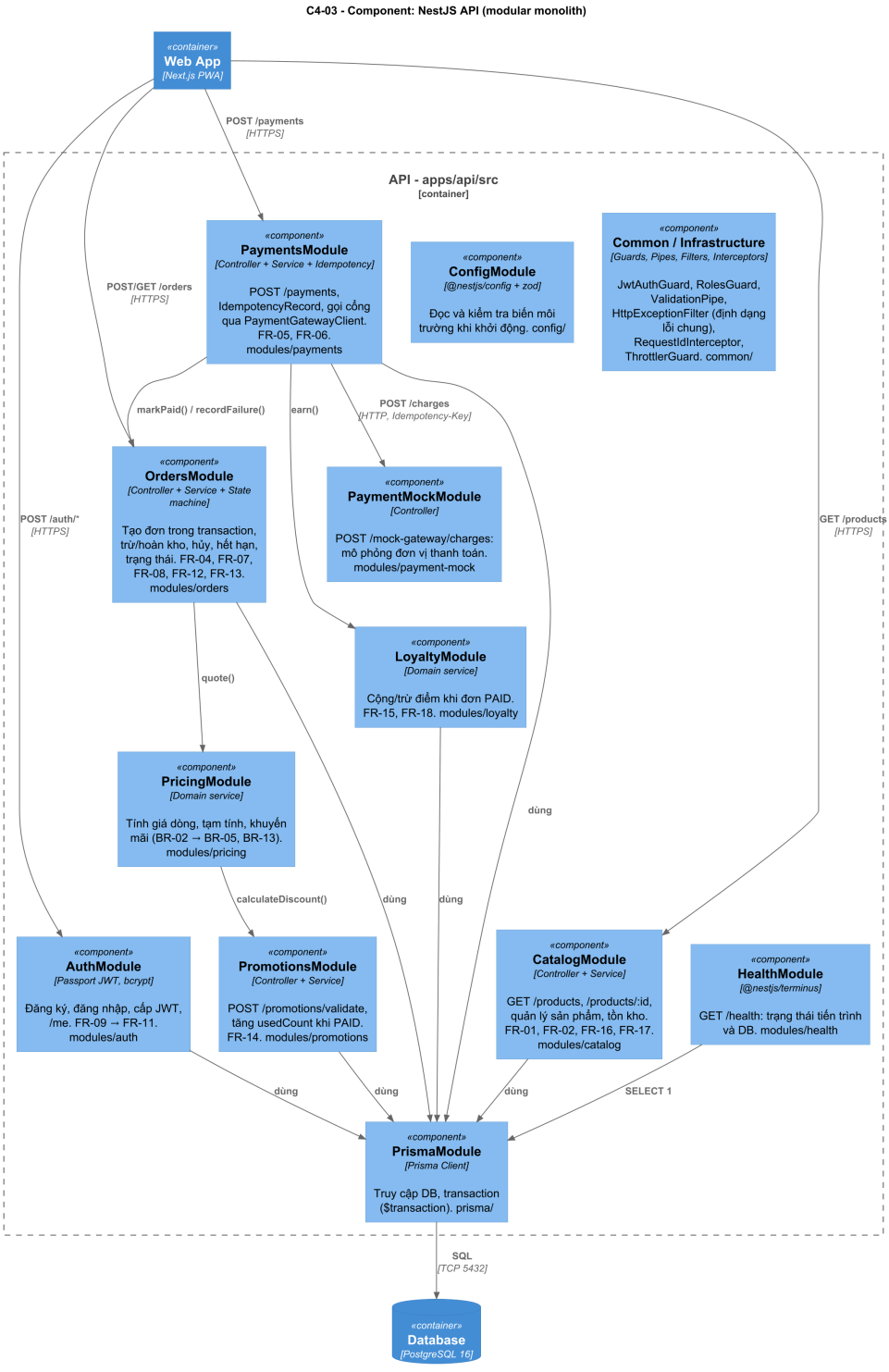
Kiến trúc được mô tả theo **mô hình C4** ở ba mức (System Context, Container, Component). Bên trong mỗi module backend áp dụng **kiến trúc phân lớp**. Mọi sơ đồ nằm trong `docs/architecture/diagrams/` (`.puml` và `.svg`); mô tả đầy đủ ở `docs/architecture/architecture.md`.



Hình 8: C4-01 – System Context của BrewLite



Hình 9: C4-02 – Container của BrewLite



Hình 10: C4-03 – Component của NestJS API (modular monolith)

4.1.2 Động lực kiến trúc

Bảng 35: Động lực kiến trúc

Mã	Động lực	Ảnh hưởng tới kiến trúc
C1	Bắt buộc Next.js, NestJS, PostgreSQL	Ba container chính cố định
C9, C11	0 đồng, một người làm	Modular monolith thay vì microservices; dùng nền tảng có gói miễn phí
NFR-13, BR-07	Không bán vượt kho	Trừ kho bằng `UPDATE ... WHERE stock >= qty` trong transaction
FR-06, BR-10, BR-11	Không trừ tiền hai lần	Bảng `idempotency_records` + unique index; Payment Service mô phỏng cũng idempotent
NFR-04 → 08	Bảo mật	JWT, bcrypt, validate whitelist, Helmet, rate limit, CORS, secret qua biến môi trường
NFR-09, NFR-11	Mobile-first, PWA	Next.js App Router + service worker
NFR-14 → 17	Health, Docker, CI/CD	`/health`, Dockerfile mỗi app, image lên GHCR, deploy tự động
NFR-19	Logging	Log JSON có `requestId`, che dữ liệu nhạy cảm

4.1.3 Các container và thành phần

Bảng 36: Container của BrewLite

Container	Công nghệ	Trách nhiệm	Mã nguồn
Web App	Next.js App Router, React, TypeScript, TanStack Query, Zustand, Serwist, Tailwind CSS	Giao diện khách, nhân viên, quản lý; PWA	`apps/web`
API	NestJS, Prisma, Passport JWT, nestjs-pino, terminus	REST `/api/v1`, nghiệp vụ, transaction	`apps/api`
Payment Service (mô phỏng)	Module NestJS, HTTP	`POST /mock-gateway/charges`	`apps/api/src/modules/payment-mock`
Database	PostgreSQL 16	Lưu trữ bền vững	`apps/api/prisma`
Shared (thư viện)	TypeScript, zod	Schema validate dùng chung	`packages/shared`

4.1.3.1 Frontend Next.js/PWA. App Router; trang menu là Server Component để tải nhanh, phần tương tác là Client Component. TanStack Query quản lý dữ liệu từ API (loading/error, retry), Zustand + `localStorage` giữ giỏ hàng, react-hook-form + schema zod dùng chung với backend. Serwist tạo service worker: precache tài nguyên tĩnh, cache menu kiểu *stale-while-revalidate*, **không cache** API đơn hàng, thanh toán, xác thực; có trang offline. `Idempotency-Key` được sinh khi bấm thanh toán và giữ trong `sessionStorage` tới khi có kết quả cuối.

4.1.3.2 Backend NestJS theo module/domain. Các module `auth`, `catalog`, `pricing`, `orders`, `payments`, `loyalty`, `promotions`, `payment-mock`, `health`, cùng `prisma`, `config`, `common`. Mỗi module phân lớp Controller → Service → Domain logic thuần → Prisma. Module chỉ gọi nhau qua service đã export; transaction dùng `prisma.$transaction` và truyền transaction client cho các service tham gia (ví dụ trừ kho khi tạo đơn).

4.1.3.3 PostgreSQL và migration. Prisma quản lý schema và migration (commit trong `apps/api/prisma/migrations`). Production chạy `prisma migrate deploy` khi container khởi động vì DB không public. Quy tắc *expand* → *contract* giúp rollback image không làm vỡ schema. Ràng buộc toàn vẹn: `CHECK (stock >= 0)`, partial unique `payments(order_id) WHERE status = 'SUCCEEDED'`, unique `idempotency_records(user_id, key)`, unique `loyalty_transactions(order_id, type)`.

4.1.3.4 Dịch vụ thanh toán mô phỏng. `PaymentMockModule` chạy cùng tiến trình API nhưng chỉ được gọi qua HTTP bằng `PaymentGatewayClient` (`PAYMENT_GATEWAY_URL`, timeout 10 giây, header `Idempotency-Key`); lưu giao dịch vào bảng `mock_charges` để idempotent cả khi khởi động lại.

4.1.3.5 Logging, health check và cấu hình. `nestjs-pino` ghi log JSON có `requestId` và che `authorization`, `password`, số thẻ; `GET /health` ping DB, trả 200 hoặc 503; `ConfigModule` kiểm tra biến môi trường bằng zod khi khởi động (fail fast); `.env.example` liệt kê đủ biến.

4.1.4 Quyết định kiến trúc (ADR)

Mỗi quyết định được ghi thành ADR ngắn trong `docs/architecture/adr/` theo mẫu Bối cảnh – Quyết định – Phương án đã cân nhắc – Hệ quả.

Bảng 37: Tóm tắt ADR: lựa chọn và trade-off

ADR	Quyết định	Được	Mất / hạn chế
001	Monorepo pnpm (`apps/web`, `apps/api`, `packages/shared`)	Schema dùng chung, một PR cho thay đổi xuyên tầng	Dockerfile build từ root; CI cần cache
002	NestJS modular monolith, phân lớp theo module	Transaction DB cục bộ đúng và đơn giản; dễ unit test	Scale cả khối; cần kỷ luật ranh giới module
003	Next.js App Router + PWA (Serwist)	SSR nhanh, cài được, có trang offline	Service worker có thể giữ bản cũ; không được cache API đơn hàng
004	PostgreSQL + Prisma, migrate khi container khởi động	Migration có version; không cần mở DB ra ngoài	Phải tuân thủ expand/contract
005	Payment mock cùng tiến trình, ranh giới HTTP	0 chi phí, không cold start riêng, giữ hợp đồng HTTP + idempotency	Không độc lập về lỗi với API
006	JWT Bearer HS256, 60 phút, bcrypt	Đơn giản, hợp CORS khác domain	Rủi ro XSS nếu lộ token; không thu hồi sớm
007	Polling 3 giây cho trạng thái đơn	Chạy ổn mọi nền tảng, dễ test, đạt ≤ 5 giây	Nhiều request hơn SSE
008	pino JSON, `/health`, config zod	Truy vết theo `requestId`; fail fast	Log chỉ giữ theo gói miễn phí
009	Vercel + Render + GHCR	HTTPS, CDN sẵn; DB nội bộ; rollback theo SHA	Cold start; Postgres free hết hạn 30 ngày

4.1.5 Lý do chọn, ưu điểm và hạn chế

Bảng 38: Modular monolith so với microservices

Tiêu chí	Modular monolith (chọn)	Microservices
Công sức (1 người, 2 Sprint)	Thấp: một repo, một API, một DB	Cao: nhiều service, nhiều pipeline
Transaction đơn + kho + thanh toán	Transaction DB cục bộ, đúng (NFR-13)	Cần saga/outbox, khó đúng
Chi phí hạ tầng	1 web service + 1 DB	Vượt gói miễn phí
Mở rộng độc lập	Hạn chế	Tốt
Khả năng tách sau này	Có: module ranh giới rõ, Payment qua HTTP	–

Ưu điểm: đơn giản, hợp quy mô; luồng chính dựa trên transaction DB nên dễ đảm bảo không bán vượt kho và không trừ tiền hai lần; module ánh xạ 1–1 với nhóm FR; chạy local bằng một lệnh `docker compose up`.

Hạn chế: API và Payment Service mô phỏng chung tiến trình; Render free cho ngủ dịch vụ sau 15 phút (cold start 30–60 giây); polling tốn request hơn SSE; frontend và API khác domain nên cần CORS và JWT lưu phía client.

4.2 Task 3.2 – Deployment diagram

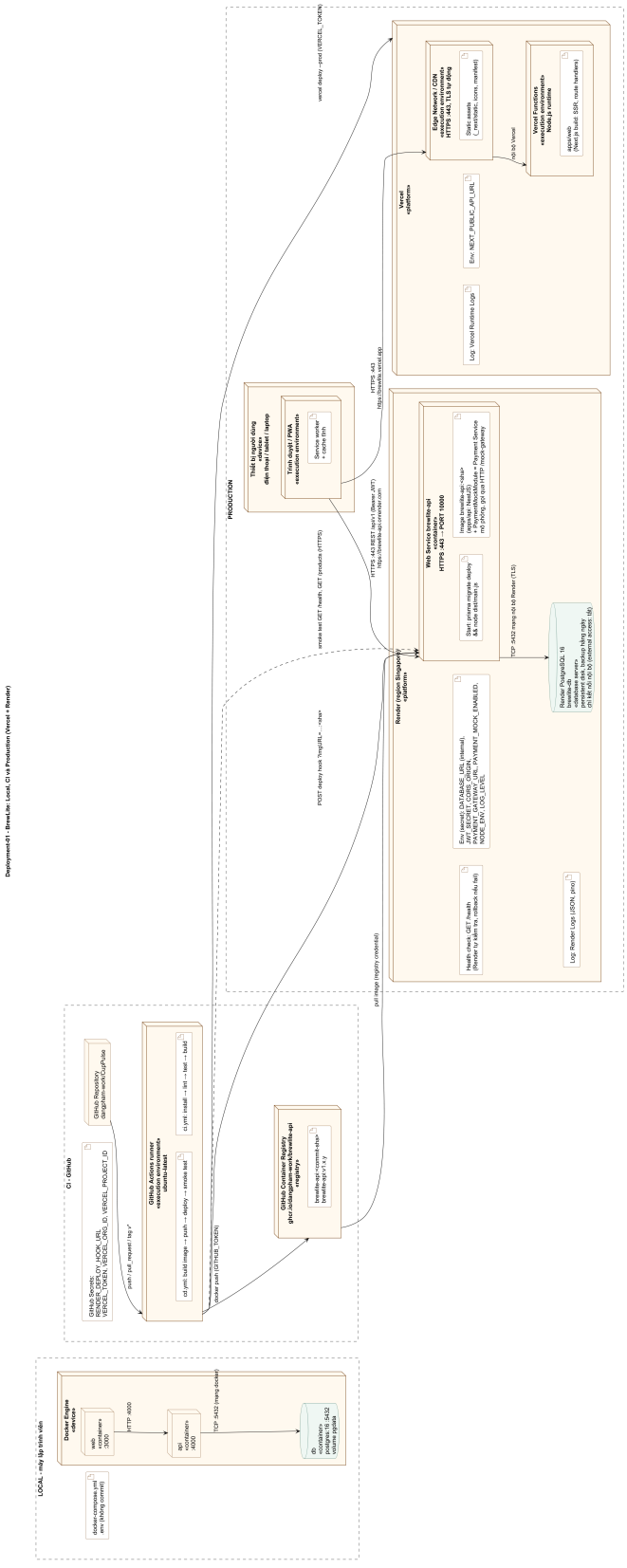
Production dùng nền tảng thực tế: **Vercel** cho Next.js, **Render** cho NestJS API (Docker image từ GHCR) và PostgreSQL, **GitHub** cho mã nguồn, CI/CD và container registry. Sơ đồ thể hiện cả ba ranh giới Local, CI và Production. Bản vector đầy đủ: `docs/architecture/diagrams/deployment.svg`.

Bảng 39: Truy vết node triển khai → kiến trúc → mã nguồn

Node / artifact	**Container C4 / thành phần**	**Mã nguồn / cấu hình**
Trình duyệt / PWA, service worker	Web App (client)	`apps/web/src/app/sw.ts`, `manifest.webmanifest`
Vercel Edge Network / CDN	Web App (tài nguyên tĩnh)	`apps/web/public/`, `.next/static`
Vercel Functions	Web App (SSR)	`apps/web/`, `vercel.json`
Render Web Service `brewlite-api`	API	`apps/api/`, `apps/api/Dockerfile`
Image `brewlite-api:<sha>`	API + PaymentMockModule	`.github/workflows/cd.yml`
Start `prisma migrate deploy`	PrismaModule, migration	`apps/api/prisma/migrations/`, `docker-entrypoint.sh`
Health check `GET /health`	HealthModule	`apps/api/src/modules/health/`
Render PostgreSQL `brewlite-db`	Database	`apps/api/prisma/schema.prisma`
GitHub Actions `ci.yml`, `cd.yml`	Quy trình CI/CD	`.github/workflows/`
GHCR `brewlite-api`	Registry của API	Tag do `cd.yml` gán
Docker Engine local (web, api, db)	Web App, API, Database	`docker-compose.yml`, `.env.example`

Bảng 40: Giao thức, cổng và kết nối

Từ → Tới	**Giao thức / cổng**	**Ghi chú**
Trình duyệt → Vercel	HTTPS 443	TLS tự động; HTTP chuyển hướng HTTPS
Trình duyệt → Render API	HTTPS 443	CORS chỉ cho domain Vercel
Render edge → container API	HTTP nội bộ, `PORT=10000`	TLS kết thúc tại Render
API → PostgreSQL	TCP 5432, mạng nội bộ, TLS	Internal Database URL; external access tắt
API → Payment Service mô phỏng	HTTP (`PAYMENT_GATEWAY_URL`)	Timeout 10 giây, `Idempotency-Key`
GitHub Actions → GHCR	HTTPS, `GITHUB_TOKEN`	`docker push`
Render → GHCR	HTTPS, registry credential	Kéo image theo SHA
GitHub Actions → Render	HTTPS, deploy hook (secret)	`?imgURL=...:<sha>`
GitHub Actions → Vercel	HTTPS, `VERCEL_TOKEN`	`vercel deploy -prod`



Hình 11: Deployment-01 – Local, CI và Production (Vercel + Render)

Bảng 41: Secret và biến môi trường

Nơi lưu	Biến	Ai đọc
GitHub Secrets	`RENDER_DEPLOY_HOOK_URL`, `VERCEL_TOKEN`, `VERCEL_ORG_ID`, `VERCEL_PROJECT_ID`	`cd.yml`
Render Environment	`DATABASE_URL` (internal), `JWT_SECRET`, `CORS_ORIGIN`, `PAYMENT_GATEWAY_URL`, `PAYMENT_MOCK_ENABLED`, `NODE_ENV`, `LOG_LEVEL`	Container API
Vercel Environment	`NEXT_PUBLIC_API_URL`	Build Next.js
`.env` local (không commit)	Như `.env.example`	Docker Compose

Health check, log, rollback. Render gọi `GET /health`; bản deploy mới không đạt thì bản cũ tiếp tục chạy. Sau deploy, CI chạy smoke test `GET /health` và `GET /api/v1/products` có retry. Log API (pino JSON) xem ở Render Logs, log web ở Vercel Runtime Logs. Rollback API bằng cách gọi lại deploy hook với image SHA trước đó; rollback web bằng `vercel rollback`. Quy trình chi tiết sẽ nằm ở `docs/deployment.md` (Task 5).

Giới hạn nền tảng. Render free ngủ sau 15 phút (cold start, R8): smoke test có retry, "đánh thức" trước buổi chấm. Render PostgreSQL free hết hạn sau 30 ngày (R1): nâng lên gói Basic (khoảng 6 USD/tháng) trước hạn hoặc tạo lại DB rồi migrate + seed.

5 Task 4 – Triển khai Sprint 1

Sprint Goal: có repository, tài liệu có kiểm soát phiên bản, CI tự động trên pull request và MVP xem menu chạy được (API, màn hình, dữ liệu seed). Sprint kéo dài 2 tuần (02/11 – 15/11/2026), gồm 9 mục backlog PB-01 → PB-09 (22 điểm). Chi tiết: `docs/process/sprint-1.md`.

5.1 Task 4.1 – Thiết lập repository

Repository GitHub https://github.com/dangpham-work/CupPulse tổ chức theo monorepo pnpm (ADR-001):

```
apps/web/              Next.js 15 mobile-first/PWA
apps/api/              NestJS 11: src/modules/<domain>, prisma/, test/
packages/shared/       Kieu du lieu va ham dung chung web <-> API
docs/                  requirements/, diagrams/, architecture/, process/
tests/                 Integration/E2E xuyen ung dung (Sprint 2)
.github/workflows/     ci.yml
docker-compose.yml     .env.example    README.md    CONTRIBUTING.md
```

Bảng 42: Yêu cầu Task 4.1 và hiện thực

Yêu cầu	Hiện thực
Cấu trúc `apps/web`, `apps/api`, `docs`	Monorepo pnpm workspace; thêm `packages/shared` cho kiểu dữ liệu dùng chung
Chiến lược branch, pull request	GitHub Flow: `main` được bảo vệ, nhánh `feat/`, `fix/`, `docs/`, `ci/`; mọi thay đổi qua PR có template (mục đích, **lý do**, mã yêu cầu, cách kiểm tra)
`README.md`, `.gitignore`, `.env.example`	README hướng dẫn chạy local (Docker Compose hoặc pnpm), test, build; `.env.example` liệt kê đủ biến, không chứa secret thật
Issue board / Product Backlog	26 issue PB-01 → PB-26 trên GitHub Issues, milestone `Sprint 1`, `Sprint 2`, nhãn `type:*`, `priority:*`, `area:*`; bản gốc `docs/process/product-backlog.md`
Quy ước commit, Definition of Done	Conventional Commits (`feat(api): ...`); DoD 8 tiêu chí trong `CONTRIBUTING.md`

Definition of Done (tóm tắt): đạt tiêu chí chấp nhận trong SRS; có test tự động; lint, typecheck, test, build pass và CI xanh; validate đầu vào ở backend; dùng tốt ở 360 px; không lộ secret; cập nhật ma trận truy vết và tài liệu; merge qua PR.

5.2 Task 4.2 – Quản lý tài liệu

Toàn bộ tạo phẩm Task 1–3 nằm trong `docs/`, có mục lục và bảng lịch sử thay đổi ở `docs/README.md`. Mỗi task vào `main` bằng một pull request riêng, mô tả rõ lý do:

Bảng 43: Lịch sử thay đổi tài liệu qua pull request

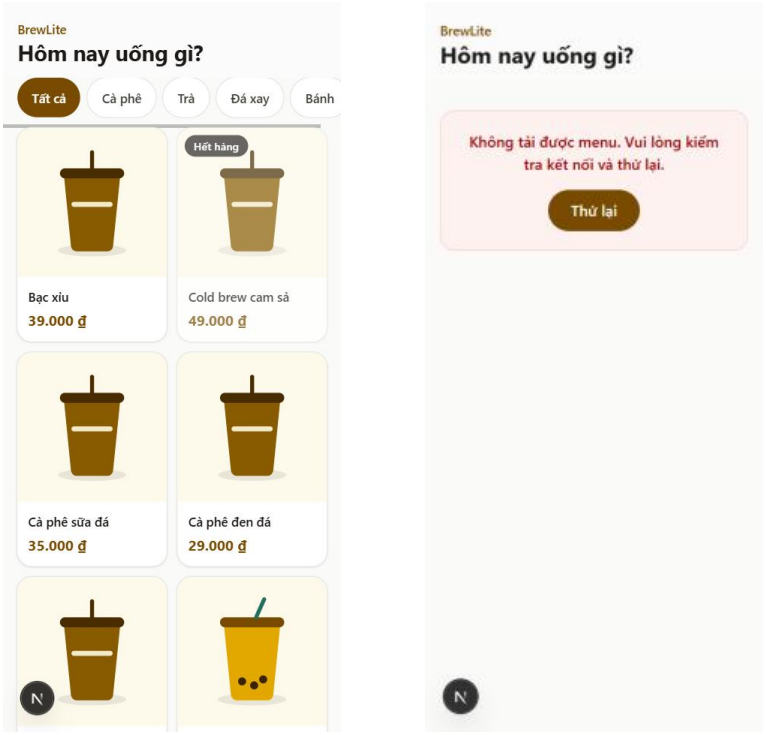
PR	Nhánh	Nội dung và lý do
#1	`docs/task1-requirements`	Task 1: bối cảnh, SRS, use-case – khởi tạo yêu cầu
#2	`docs/task2-modeling`	Task 2: activity, sequence, class, state diagram – mô hình hóa từ SRS
#3	`docs/task3-architecture`	Task 3: C4, 9 ADR, deployment diagram – thiết kế trước khi hiện thực
#30	`feat/task4-sprint1`	Task 4: mã nguồn Sprint 1, CI, quy trình; ma trận truy vết v0.4 – hiện thực FR-01, FR-02 (API)

Thay đổi quyết định kiến trúc được ghi bằng ADR mới; ADR cũ chỉ đổi trạng thái *Superseded*.

5.3 Task 4.3 – MVP Sprint 1

Bảng 44: Phạm vi MVP Sprint 1

Hạng mục	Hiện thực	Mã nguồn
`GET /products`	Món `isActive = true`; `inStock = stock > 0`; lọc `?category=`; không cần token	`apps/api/src/modules/catalog/`
`GET /products/:id`	Kèm `sizes`, `toppings`; 404 `PRODUCT_NOT_FOUND`; 400 khi id không phải UUID	`catalog.controller.ts`, `catalog.service.ts`
Lỗi và validate	Định dạng `{statusCode, code, message}`; query lạ bị từ chối (whitelist); lỗi hệ thống không lộ chi tiết	`common/`, `app.setup.ts`
Màn menu mobile-first	Lưới 2 cột ở 360 px, tab danh mục, nhãn "Hết hàng", vùng chạm $\geq$ 44 px	`apps/web/src/features/menu/`
Loading / empty / error	Skeleton; "Chưa có món nào"; thông báo lỗi + nút "Thử lại" gọi lại API	`MenuView.tsx`
Dữ liệu mẫu	Migration Prisma (kèm CHECK giá, tồn kho ≥ 0) và seed idempotent: 4 danh mục, 13 sản phẩm (1 hết hàng, 1 đã ẩn), 4 topping	`apps/api/prisma/`
Health check	`GET /health` ping DB, trả 200 hoặc 503	`modules/health/`
Chạy local	`docker compose up -build`: web, API (tự migrate + seed), PostgreSQL	`docker-compose.yml`, `apps/*/Dockerfile`



Hình 12: Màn menu ở chiều rộng mobile: dữ liệu từ API (trái) và trạng thái lỗi khi API không phản hồi (phải)

Kiểm thử Sprint 1

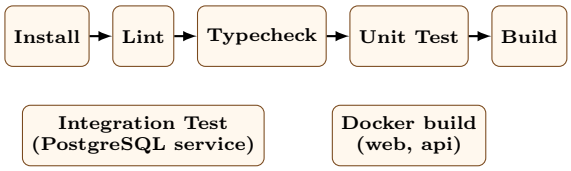
Bảng 45: Test tự động Sprint 1

Mã	Loại	Nội dung	Số ca
UNIT-T07, T08	Unit (Jest)	`CatalogService`: lọc, ánh xạ `inStock`, topping ngừng bán, `PRODUCT_NOT_FOUND`; kiểm tra biến môi trường	8
API-T01, T02	API (Jest + supertest, Prisma giả lập)	200 không token, lọc danh mục, query lạ → 400, id sai → 400, 404, lỗi DB → 500, `/health` 200/503	9
INT-T08	Integration (PostgreSQL thật)	Migrate + seed từ DB trống; 12 món đang bán, size, topping, món ẩn → 404	5
UI-T01	Component (Vitest + Testing Library)	Skeleton, danh sách, empty, lỗi + Thử lại, mất mạng, "Hết hàng", tab danh mục	7
UNIT-T09	Unit (node:test)	`formatVnd` theo quy ước `45.000 ₫`	2
		Tổng	**31**

Test được viết trước phần hiện thực. Để chắc test thật sự kiểm tra hành vi, nhóm cố ý làm sai logic (đổi `stock > 0` thành `stock >= 0`, đổi nhãn "Hết hàng") và xác nhận test tương ứng chuyển sang thất bại.

5.4 Task 4.4 – Continuous Integration

Workflow `.github/workflows/ci.yml` chạy khi mở hoặc cập nhật pull request vào `main` và khi push lên `main`:

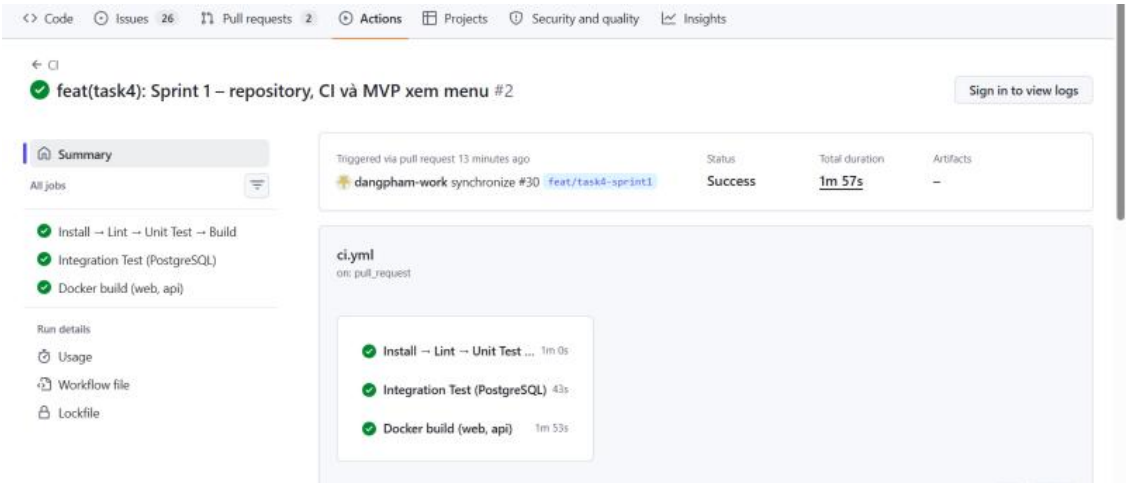


Job *quality* chạy tuần tự Install → Lint → Typecheck → Unit Test → Build; job *integration* dựng PostgreSQL 16 làm service container, chạy migrate, seed hai lần (kiểm tra seed idempotent) rồi integration test; job *docker* build image web và API để kiểm tra Dockerfile. **Branch protection** của `main` yêu cầu cả ba job pass mới được merge (NFR-16).

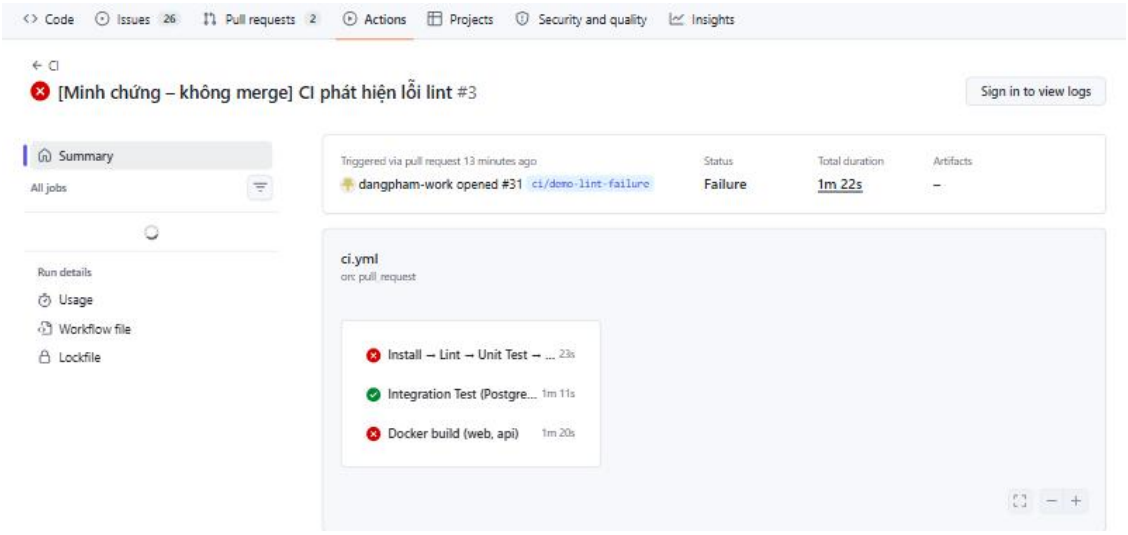
Minh chứng pipeline

Bảng 46: Các lần chạy CI của Sprint 1

Lần chạy	Commit	Kết quả	Link
PR #30 – lần 1	`96a0197`	**Cancelled**: commit mới được push nên workflow tự hủy lượt cũ	run 41408066493
PR #30 – lần 2	`cd3f53d`	**Success**: quality 1m 0s, integration 43s, docker 1m 53s	run 41408203576, PR #30
PR #31 (minh chứng lỗi)	`89d5753`	**Failure** ở bước **Lint**: biến không dùng; Typecheck, Unit Test, Build bị bỏ qua; job Docker cũng đỏ vì `next build` chạy ESLint	run 41408237890, PR #31 (đóng, không merge)



Hình 13: CI của PR #30 thành công: ba job xanh



Hình 14: CI của PR #31 phát hiện lỗi lint: job chất lượng và job Docker đỏ

Trích log bước Lint của run 41408237890:

```
apps/web lint:   27:7  error  'unusedForCiDemo' is assigned a value but never used
                 @typescript-eslint/no-unused-vars
apps/web lint: x 1 problem (1 error, 0 warnings)
```

Sự cố trong Sprint. Mẫu `.gitignore` Python có dòng `lib/` làm `apps/web/src/lib/api.ts` không được commit ở lần push đầu (máy local vẫn build được vì tệp có trên đĩa). Lỗi được phát hiện khi chuẩn bị nhánh minh chứng, trước khi lượt CI đầu chạy xong, và được sửa ở commit `cd3f53d`. Bài học: CI trên máy sạch là nơi kiểm chứng repository đầy đủ, nên luôn chờ CI xanh trước khi merge.

6 Task 5 – Triển khai Sprint 2, CI/CD và Delivery

Phần này sẽ được bổ sung khi thực hiện Task 5: hoàn thiện MVP, kiểm thử, Continuous Delivery/Deployment, triển khai production và bàn giao.

7 Ma trận truy vết

Ma trận truy vết (`docs/traceability-matrix.md`, phiên bản 0.4 sau Task 4) liên kết mỗi yêu cầu với use-case/user story, sơ đồ thiết kế, API và test. Dòng có dấu (*) đã hiện thực ở Sprint 1: cột Code/API trỏ tới mã nguồn, cột Test là test đang chạy trong CI. Các dòng khác là hợp đồng API từ SRS và mã test dự kiến, cập nhật ở Task 5.

Quy ước mã test: `UNIT` unit test; `API` integration test một endpoint; `INT` integration test nhiều bước; `E2E` end-to-end; `SMOKE` smoke test sau triển khai.

Bảng 47: Ma trận truy vết yêu cầu chức năng

Mã yêu cầu	UC / User Story	Thiết kế / Sơ đồ	Code / API	Test
FR-01 (*)	US-01 Xem menu, UC-02	Activity-01, Class-01, C4-03 Catalog	`GET /products`; `catalog.controller.ts`, `MenuView.tsx`	API-T01, UNIT-T07, INT-T08, UI-T01
FR-02 (*)	US-02, UC-02	Activity-01, Class-01	`GET /products/:id`; `catalog.service.ts`	API-T02, INT-T08
FR-03	US-03, UC-03	Activity-01	Giỏ hàng phía client	UNIT-T02, E2E-T01
FR-04	US-05, UC-01	Activity-01, Sequence-01, Class-01, State-01, C4-03 Orders, ADR-002	`POST /orders`	INT-T01, INT-T02, INT-T06
FR-05	US-05, UC-01 Thanh toán	Sequence-01, Class-01, State-01, C4-03 Payments, ADR-005	`POST /payments`	INT-T03, INT-T04
FR-06	US-05, UC-01	Sequence-01, Class-01, ADR-004, ADR-005	`POST /payments` + `Idempotency-Key`	INT-T05
FR-07	US-06, UC-06	Activity-01, Sequence-01, State-01, ADR-007	`GET /orders/:id`	API-T03, E2E-T01
FR-08	US-07, UC-07	Class-01	`GET /orders`	API-T04
FR-09	US-04, UC-04	Class-01	`POST /auth/register`	API-T05
FR-10	US-04, UC-05	Activity-01, Sequence-01, C4-03 Auth, ADR-006	`POST /auth/login`	API-T06, UNIT-T03
FR-11	Mọi UC	Class-01 (Role)	Guard `/staff/*`, `/manager/*`	API-T07
FR-12	US-10, UC-10	Activity-01, State-01, Class-01	`GET/PATCH /staff/orders`	API-T08, UNIT-T04
FR-13	US-12, UC-12, UC-15	Activity-01, State-01	`POST /orders/:id/cancel`	INT-T07
FR-14	US-08, UC-08	Sequence-01, Class-01	`POST /promotions/validate`	UNIT-T05
FR-15	US-09, UC-09	Sequence-01, Class-01	`GET /me/loyalty`	INT-T03, UNIT-T06
FR-16	US-11, UC-11	Class-01	`/manager/products`	API-T09
FR-17	US-01, UC-02	Activity-01	`GET /products?q=`	API-T10
FR-18 → 20	Could	Class-01 (một phần)	Chỉ khi thực hiện	Chỉ khi thực hiện
FR-21 → 24	Won't	–	Không có	Không áp dụng

Bảng 48: Ma trận truy vết yêu cầu phi chức năng then chốt

Mã yêu cầu	Liên quan	Thiết kế / Sơ đồ	Code / Cấu hình	Test / Minh chứng
NFR-04	FR-10	Sequence-01 (JwtAuthGuard), ADR-006	Module auth	API-T06
NFR-05	Mọi API	Sequence-01 (ValidationPipe)	ValidationPipe toàn cục	API-T01 → T10
NFR-07	–	Deployment-01 (HTTPS, DB không public), ADR-009	Cấu hình Vercel, Render	Kiểm tra cấu hình
NFR-11	FR-01	C4-02 Web App, ADR-003	`apps/web` (Serwist)	Lighthouse PWA
NFR-13	FR-04, FR-05	Sequence-01 (Transaction), ADR-002, ADR-004	Service đơn hàng, thanh toán	INT-T06
NFR-14	–	Deployment-01 (health check), ADR-008	`GET /health`	SMOKE-T01
NFR-16	–	Deployment-01 (`ci.yml`), ADR-001	`.github/workflows/ci.yml`	Log CI
NFR-17	–	Deployment-01 (`cd.yml`, GHCR), ADR-004, ADR-009	`.github/workflows/cd.yml`	Log CD, SMOKE-T01
NFR-19	–	C4-03 Common, ADR-008	`nestjs-pino`	Log production

Truy vết từng node của Deployment-01 tới container C4 và mã nguồn: xem Bảng 39.

Tài liệu tham khảo

[1] Khoa Công nghệ Thông tin, Trường Đại học Sài Gòn, *BrewLite – Assignment 2: Requirements to Deployment*, Ver 1.0, 2026.

[2] IEEE, *IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications*, 1998.

[3] A. Cockburn, *Writing Effective Use Cases*, Addison-Wesley, 2000.

[4] PlantUML, "**link: https://plantuml.com**", lần truy cập cuối: 01/11/2026.